Applied Intelligence (2024) 54:3756–3785
https://doi.org/10.1007/s10489-024-05356-7

Graph-based substructure pattern mining with edge-weight

Md. Ashraful Islam[1] · Chowdhury Farhan Ahmed[1] · Md. Tanvir Alam[1] · Carson Kai-Sang Leung[2]

Accepted: 21 February 2024 / Published online: 8 March 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract
To represent complex inter-relationships among entities, weighted graphs are more useful than their unweighted counterparts. In a transactional graph setting, researchers have made several attempts to mine weighted frequent subgraphs from a collection of edge-weighted graphs, which will serve as the representative feature of the underlying graph database and can be further used for analysis. As weighted support of any pattern does not hold downward closure property, a property that is often used in frequent pattern mining to control search space, has made weighted frequent substructure mining a tremendously difficult task. This article proposes an efficient weighted frequent subgraph mining framework called WFSM-MaxPWS for graphs with static edge weights. We introduce a new pruning technique called MaxPWS pruning along with canonical labeling of subgraphs, which helps reduce the search space significantly without compromising completeness. Extending the WFSM-MaxPWS framework, we propose another framework called DewgSpan that is capable of mining graphs with dynamic edge weight. DewgSpan utilizes a summarized edge-weight distribution table to overcome the new challenges of dynamic edge-weight settings. Evaluation results show that WFSM-MaxPWS and DewgSpan are significantly faster than the existing MaxW pruning technique of weighted pattern mining.

Keywords Data mining · Pattern mining · Substructure mining · Weighted subgraph

1 Introduction

Frequent pattern mining is one of the significant branches of data mining that discovers frequently appeared patterns that satisfy a user-defined threshold level. As a result, many algorithms have been developed from general pattern mining (e.g., Apriori [1], FP-growth [2]) to specific pattern mining (e.g., mining sequential patterns [3, 4], weighted patterns [5], uncertain patterns [6, 7], data streams [8], etc.). Patterns are not always a set of trivial items; more often, they have structural or sequential properties that should be retained when representing them, for example, biological sequences, semi-structured texts like HTML or XML, chemical compounds, RNA secondary structures, etc. [9]. Moreover, to deal with such complex structured data, graph mining [10–13] has emerged as an extensively valuable area as through graphs, we can store the structure of data and the value that can be exploited to represent complex objects.

For the processing of graph objects like graph clustering or classification [14–17], one needs to feed graphs to the processing model by creating a feature vector, which is often in binary feature space containing information about the presence or absence of subgraphs in a particular graph. As the number of subgraphs depends exponentially on a graph's vertex count and edge count, representing a graph by all its possible subgraphs is prone to the high intrinsic dimension of feature space. **F**requent **S**ubgraph **M**ining (**FSM**) [18, 19] solves the problem by identifying frequently occurred subgraph patterns in an attempt to reduce feature vector dimension. For example, in [20], frequent subgraphs mined from a collection of graphs are used to learn feature vectors for the graphs. These vectors are used later for both graph classification and clustering. Another approach proposed in [21] mines frequent subgraphs and then filters out

✉ Md. Ashraful Islam
ashraful@du.ac.bd

Chowdhury Farhan Ahmed
farhan@du.ac.bd

Md. Tanvir Alam
tanvircsedu15@gmail.com

Carson Kai-Sang Leung
kleung@cs.umanitoba.ca

[1] University of Dhaka, Dhaka, Bangladesh

[2] University of Manitoba, Winnipeg, Manitoba, Canada

the non-discriminating subgraphs. The final resultant subgraphs are utilized as features to learn vector representations for graph classification. Compared to unweighted graphs, *weighted graphs* have strong representational power because different importance can be imposed on different relations belonging to the structure of data. Moreover, *weighted frequent subgraphs* can describe the underlying graph database more accurately by reflecting the interestingness of the pattern through weight when compared to frequent unweighted patterns, and thus can contribute significantly in areas like feature extraction for graph classification, association rule mining, and graph clustering. This observation has given birth to a new research area: **W**eighted **F**requent **S**ubgraph **M**ining (**WFSM**).

FSM is also used in neural network architecture creation where commonly occurring architecture in existing neural network applications can be mined and used to construct a new one for similar problems. But, each existing neural network may not have equal importance as each varies in efficiency, accuracy, and similarity with the problem in consideration. Moreover, all inter-layer connections also have different significance. Instead of using plain FSM, WFSM should give better results as it considers inter and intra-model connection significance. Image analysis can also benefit from weighted substructure mining [22]; local image features are considered nodes, and geometric relations between them are described as edge weights, and then WFSM methods can be applied. In software engineering, control flow graphs (CFG) are used for behavioral fault localization [23] where WFSM plays a vital role in quickly pinpointing future bugs. Another similar application of WFSM is malware call graph analysis [24, 25]. Previously, reverse engineering tools were applied on malware and benign executables to identify semantic juice [26] of each basic block of call graphs and then frequently occurred substructures in call graphs were identified using plain FSM to be used as a signature in malware detection. But, nowadays, malware creators insert tons of benign codes into their malware to evade detection. These types of malware are known as metamorphic malware. The insertion of huge chunks of benign code in malware binaries makes it impossible to identify sophisticated malware using signature-based approaches with plain FSM. However, no matter how much benign code is inserted, original suspicious functions still exist in the malware code. Some commonly encountered Windows functions by malware analysts are LdrLoadDll, LoadResource, LsaEnumerateLogonSessions, MapViewOfFile, MapVirtualKey, NetScheduleJobAdd, BitBlt, etc. So, if we give higher weights to bigrams in call graphs containing functions like these according to their importance, we can evade highly frequent benign call sequences in malware feature extraction. Figure 1 shows a weighted malware callgraph of the Bifrose variant. Observed from the figure, through reverse engineering on the binary executable of this malware, six different branchings have been discovered where to call the external function *GetCommandLineA* has been associated with a high weight value as it indicates the process will be asking for terminal access which is highly suspicious. Call to memory management-related external functions (*HeapAlloc, GetProcessHeap*) also have high weight values. However, the call to the local function *sub_407D1C* (function names starting with sub indicate local unnamed functions implemented by the programmer) is associated with even higher risk (high weight value) after analyzing its sandbox behavior. A two-hop call to the same function is also assigned with high risk for the same reason. The task of detecting such metamorphic malware, thus, can take significant benefit from WFSM by giving higher weights to those call sequences. In short, weighted frequent subgraphs have the potential to summarize more interesting and useful information than unweighted counterparts. Besides, to improve detection accuracy, static feature (call graphs) is often attributed to behavioral features where edge weights between basic blocks usually denote the number of times jmp/call occurred between them in a particular test run which leads to call graphs with dynamic edge weight and requires WFSM to be capable to handle it.

Analyzing anticancer drugs is another crucial application of WFSM. Poor performance of cancer drugs calls for the discovery of novel drugs. Anticancer drug screening is such a preselection method to invent effective cancer drugs. Researchers often use the effectiveness of different chemical substances on various strains of mutant yeast cells for this test. One such frequently tested species is *Saccharomyces cerevisiae*.

Growth inhibition of its mutant strains is screened to test the drug's effectiveness. To invent an effective novel drug, discovering the chemical substructure that contributes most to its effectiveness plays a crucial role, and weighted substructure mining does exactly that. One such dataset is NCI Yeast Anticancer Drug Screen for the bub3 strain [1]. A cleaned version of this dataset will be used for performance analysis in Section 6.2. Figure 2 shows chemical structures of two substances. The screen was performed in three stages: stage-0 (single dose, 50 microM), stage-1 (two doses, 5 microM & 50 microM) and stage-2 (five doses, 100 microM, 33 microM, 11 microM, 3.7 microM & 1.2 microM). If growth inhibition is more than 90% compared to untreated cases, the drug is marked as effective. More efficacious chemical structures are discovered iteratively by combining the frequently occurring substructure in the drugs that showed better growth inhibition. This new phase requires incremental mining from the set of graphs that have drifted from their original structure. However, incremental mining is beyond the scope of this article.

[1] https://pubchem.ncbi.nlm.nih.gov/bioassay/167

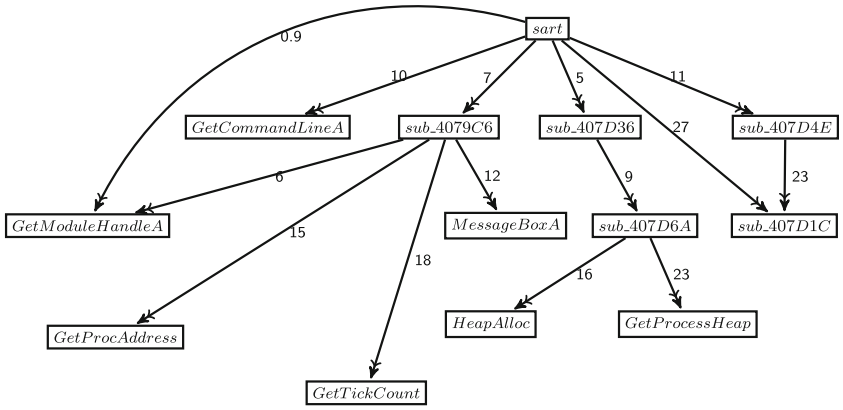


Fig. 1 Malware Callgraph (Bifrose Variant)

Existing algorithms handle general weighted frequent itemset mining in FP-tree-based mining algorithms by first considering GMaxW (weight of maximum weighted item from initial global FP-tree) and then recursively considering LMaxW (weight of the maximum weighted item in conditional FP-trees) as the maximum possible itemset weight for pruning [27]. The completeness is ensured by sorting itemsets in weight ascending order and extending a pattern by the next highest weighted item. This approach generates a moderate number of candidate patterns because recursively lighter weights are considered for pruning.

But, unlike itemset extension, a subgraph can only be extended by a new edge with at least one vertex already present in the current subgraph. For example, in graph $G4$ of Fig. 3, subgraph: $\{a - a\}$ with weight 1.9 can only be extended by edge $\{a - c\}$ with weight 0.75 although edge $\{c - c\}$ has a higher weight value of 1.8. Edge $\{c - c\}$ can be considered the extension of $\{a - a - c\}$. So, sorting edges of the graph would not ensure that higher weighted edges will be extended first, which removes the opportunity of recursively replacing the maximum possible weight of the extended subgraph with LMaxW. Thus, pruning using only GMaxW, i.e., MaxW, is possible. However, as MaxW is a pretty high overestimation of the actual average weight of the subgraph, many unnecessary candidate subgraphs are generated, thus leading to a long runtime. Although attempts to reduce the number of generated candidate sets have been made, most of the existing approaches either use affinity-based pruning (which usually imposes extra conditions to measure the interestingness of a subgraph) or other alternative weighting techniques (which redefines subgraph weights in such a way that they hold downward closure property).

This article aims to reduce the number of generated candidate sets via our non-trivial adoption of affinity-based conditions and subgraph weighting techniques. The work is inspired by our observation that, when considering MaxW

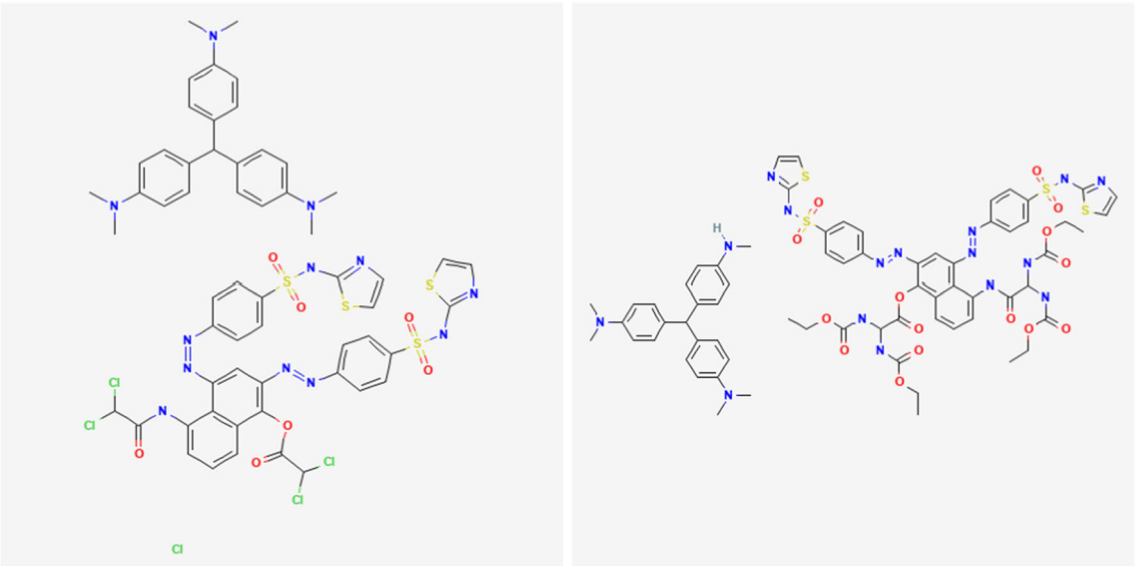


Fig. 2 Sample Substance Chemical Structures from Yeast Datatset

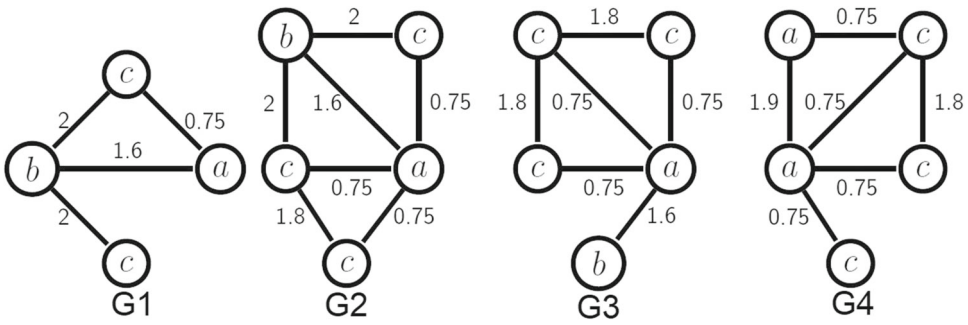


Fig. 3 Sample graph database sGDB with static edge weight

as the maximum possible weight of an extended subgraph, those already-seen average weights of the subgraph are often ignored. So, we use (i) these already-seen weights of a subgraph and (ii) some statistical information about the dataset to calculate the maximum possible weight and frequency for extensions of the subgraph up to many edges. Consequently, the **Max**imum **P**ossible **W**eighted **S**upport (**MaxPWS**) can be computed for a subgraph. Moreover, by making a minor but intelligent change in canonical ordering for weighted subgraphs, the magnitude of MaxPWS can further be brought closer to the actual weighted support, which leads to safe and effective pruning of unnecessary candidate subgraphs. Here, we propose two WFSM algorithms, named *WFSM-MaxPWS* and *Dewgspan*, for discovering all the **W**eighted **F**requent **S**ubgraph **P**atterns (**WFSP**) ensuring completeness.

In WFSM-MaxPWS, we consider static weights of edges only. That is, we focus on the condition that *"All edges with same edge label and end-point node label have the same weight"*. DewgSpan, on the other hand, relaxes this condition and can handle graphs with dynamic edge weights. DewgSpan uses weight-range wise compressed **W**eight **D**istribution **T**able (**WDT**) to calculate pruning measure $MaxPWS$ for graphs with dynamic edge weight. Although DewgSpan is also inherently capable of handling graphs with static edge weight, WFSM-MaxPWS performs comparatively better than DewgSpan.

Hence, *our key contributions* of this article are as follows:

- a tighter pruning technique **MaxPWS** for weighted subgraph mining with static and dynamic edge weight, and
- canonical ordering for weighted subgraphs that makes MaxPWS even tighter for graphs with static edge weight,
- a weighted frequent subgraph mining algorithm for called **WFSM-MaxPWS**, which uses the MaxPWS pruning technique in static edge weight setting.
- a weighted frequent subgraph mining algorithm for called **DewgSpan**, which uses the MaxPWS pruning technique in a dynamic edge weight setting.

In Section 2, we describe other closely related approaches of FSM and WFSM that we partially use in our framework or studied similar problem scope. Sections 4 and 5 are dedicated to the description of our proposed framework, and Section 6 discusses experimental results and analysis. Finally, we draw conclusions regarding our theoretical and experimental findings by briefly discussing future works.

2 Related works

Pattern-growth-based frequent subgraph discovery algorithm gSpan [18] has set the platform for diverse FSM approaches like uncertain subgraph mining [28], temporal subgraph mining [29, 30], subgraph mining in a multilevel directed graph [31] or in distributed environment [32] along with weighted subgraph mining. The gSpan algorithm follows a depth-first search approach to discover **F**requent **S**ubgraph **P**atterns (**FSP**) from transactional graph database starting with an empty subgraph and then gradually extending it with different edges. The gSpan algorithm represents each subgraph using DFScode – the linear order of edges for depth-first traversal. To avoid duplicate discovery/extension of the same graph, gSpan proposed to follow a canonical order, namely the right-most-path extension policy. Right-most-path extension policy, although it does not guarantee to prevent duplicate discovery, gives way to ranking them using an extended tuple comparison rule and among different isomorphic instances, one with the lowest rank is said to be *canonical* and is kept and extended. In this article, we modify such an extended tuple comparison rule for *weighted* graphs to discover **W**eight **F**requent **S**ubgraph **P**atterns (**WFSP**). A challenge is that such modification affects the rank and thus changes the canonical DFScode of a subgraph.

Eichinger et al. [33] proposed to use edge-weighted call graphs to localize non-crashing occasional bugs with a combination of structural and numerical techniques. They argued that the raw call graph contains call frequency, and unweighted FSM approaches must pay attention to this vital feature, thus giving less significant patterns. They proposed to find frequent subgraphs first and then apply weight constraints to identify apt patterns. This method will likely lose crucial patterns in the first phase of mining that have low

frequency but high weight. In image analysis, the necessity of weighted substructure mining was acknowledged by Jiang and Coenen [34] as some image objects are more important than others while classifying them. Hence, they defined the weight of an edge e as

$$\omega(e) = \frac{\tau(e)}{\sum_{1<i<n} K(G_i)} \tag{1}$$

where $\tau(e)$ is number of occurrences of edge e in the graph database, $K(G_i)$ is the number of edges in graph G_i. From the equation, it is perceivable that the weight of an edge e is the same in all its occurrences; that is, edges have static weights. Nevertheless, in the raw form, different occurrences of an edge in different locations can have different importance. *DewgSpan* can find significant patterns from such dataset, directly preserving original, inherent information.

Shinoda et al. [35] introduced a method to mine from externally and internally weighted graph databases, for example, web access log analysis of users' web sessions where vertex weight is the period user spent on a particular page (vertex) and edge weight is the number of transitions in a particular web session from one page to another. Both of these weight values serve as an internal weight, whereas the importance of the user is the external weight. The internal weight of a graph G_i is defined as

$$iw_G(G_i) = \sum_{x \in V_{G_i} \cup E_{G_i}} \omega(x) \tag{2}$$

and the internal weight of the database is defined as

$$iw_GDB = \sum_{G_i \in GDB} iw_g(G_i) \tag{3}$$

Internal weight of a pattern p in a graph g_i, $iw_g(p, G_i)$ is defined as the internal weight sum of the maximal weighted isomorphism of p in G_i that is an extension of a subgraph will always have higher weight (and consequently higher weighted support if support remains constant). As a result, the proposed approach will show biases towards large graphs, thus unnecessarily adding less important edges to patterns. This work was further extended for closed and maximal subgraph mining in [36]. Research has been conducted on internal and external utility-based graph mining [37], which also shows bias towards large subgraphs.

Eichinger et al. [38] presented a weight constraint-based substructure mining algorithm. The introduction of weight constraint gave approximate results compared to the unweighted version. However, for real-world problems like software localization, graph classification, and logistics, extracted patterns were more interesting and informative. Weight constraint also reduced search space, enabling it to execute faster. However, weight-based constraints are not anti-monotonic, they performed mining without resolving it, so their approach gives approximate results. By contrast, we ensure completeness in our proposed technique.

Another work on weighted FSM has been proposed by Jiang et al. [39], which was slightly modified for *Longitudinal Social Network Data* in [40]. Three weighting systems are proposed: Average Total Weighting (ATW), Affinity Weighting (AW) and Utility-Based Weighting (UBW). The usefulness or relevance of the result set for classification tasks was validated using these weight measures. Two datasets were used, an MRI scan dataset [41] and a text mining dataset [42]. Extracted substructures were used as a feature and yielded better classification accuracy.

In the ATW approach, the weight for a sub-graph g is calculated by dividing the sum of the average weights in graphs that contain g with the sum of the average weights across the entire data set T. Average weight of a graph G in the dataset:

$$W_{avg}(G) = \frac{\sum_{i=1}^{k} w_i}{k} \tag{4}$$

where G has k edges having weight $w_1, w_2, w_3, \ldots, w_k$. If $T = G_1, G_2, \ldots, G_t$ is the dataset having t graphs, then the total weight of this set is defined as

$$W_{sum}(T) = \sum_{i=1}^{1} W_{avg}(G_i) \tag{5}$$

and the average total weight of a subgraph is defined as

$$W_T(g) = \frac{\sum_{G_i \in \delta_t(g)} W_{avg}(G_i)}{W_{sum}(T)} \tag{6}$$

where $\delta_t(g)$ is the support set of g.

As ATW is anti-monotonic, the downward closure property holds, which is useful for pruning in pattern mining. However, the problem is that ATW gives both subgraphs with heavy-weighted edges and subgraphs with low-weighted edges the same weight if they have the same support set δ_t. For example if g_1 and g_2 in Fig. 4 have same support set ($\delta_t(g_1) = \delta_t(g_2)$), then they both will have same ATW ($W_T(g_1) = W_T(g_2)$).

AW prunes a subgraph if its edges fail to satisfy some weight correlation condition, and UBW discards a subgraph if its weight share falls below a weight-share threshold λ. In contrast, MaxPWS does not require redefining subgraph weight other than taking the average edge weights, so it can discriminate between subgraphs even if they have the same support set. Besides, MaxPWS leaves the field open to apply any application-specific affinity or utility conditions if required.

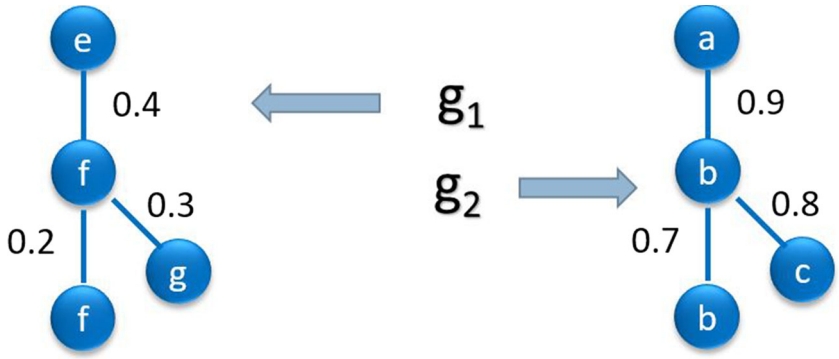


Fig. 4 Two sample subgraphs(g_1 and g_2)

To overcome the ambiguity and limitations of previous WFSM approaches, Lee et al. [43] proposed average edge weight as a subgraph weighting technique. As a result, the mining framework will not unnecessarily extend subgraphs showing biases towards large subgraphs. Besides, their mining approach is complete as they used $MaxW$ pruning measure to reduce search space which ensures no WFSP will be lost in the pruning process. Nevertheless, their $MaxW$ pruning measure that is pruning a subgraph if its support multiplied with the maximum weight ($MaxW$) for an edge in the database fails to satisfy the weighted support threshold is a considerable overestimation especially if the weight distribution is positively skewed, meaning most of the edge weights are shifted towards the lower range. Thus their search space reduction could be more efficient. Although they have described their approach in a static edge weight setting, it can also be applied with minor modifications for dynamic edge weights. We have used this algorithm, referred to as $MaxW-gSpan$, as a baseline algorithm to compare with our proposed frameworks (WFSM-MaxPWS and DewgSpan).

In [44], a length-decreasing support constraint was proposed using the weighted smallest valid extension for frequent graph mining. The rationale of this proposal is as follows: graph patterns extracted from given graph databases can have various features, which can be different for each length of the patterns. Though these affinity-based approaches significantly improve over other pruning techniques, they are not guaranteed to find all frequent weighted subgraphs. A distributed approach of weighted frequent subgraph mining was proposed in [45]. In this approach, ATW weighting technique has been adopted. Both data and computation parts have been distributed. Gupta et al. [46] proposed an approach for regular pattern mining from weighted-directed dynamic graphs with jitter. Approaches have been made to find weighted frequent subgraphs from single large graphs also [47–49]. Weighted subgraphs are mined from a single large graph where the weight of a subgraph is defined from the weight of the nodes in [47]. On the contrary, in [49], weighted subgraphs are mined from an edge-weighted single large graph. These related works focus on several special cases of weighted frequent subgraph mining, while the current article focuses on general weighted pattern mining tasks. In the preliminary conference version [50] of this work, we proposed the WFSM-MaxPWS algorithm. Our substantively new and different contributions related to the WFSM-MaxPWS algorithm include real-life motivating scenarios, all necessary, revised algorithms for implementation with a detailed explanation, step-by-step simulation on sample graph dataset, detailed analytical analysis, and extensive experimental results. Moreover, we also propose a new framework for mining WFSM from graphs with dynamic edge weights (Figs. 5 and 6).

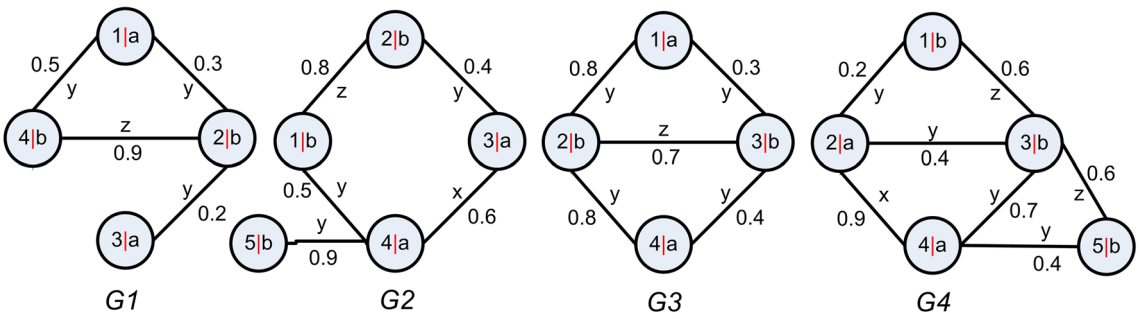


Fig. 5 Sample graph database dGDB with dynamic edge weight

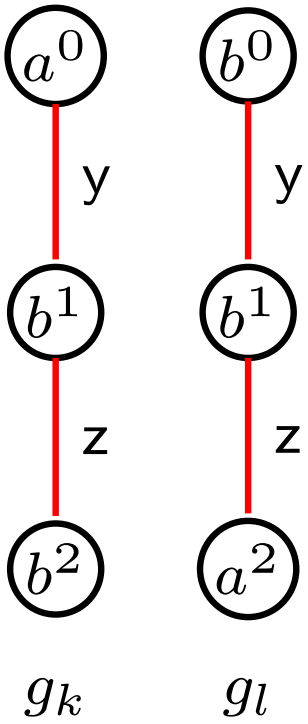


Fig. 6 Sample Subgraphs

3 Problem definition

In this section, we formally define the Weighted Frequent Subgraph Mining problem and discuss preliminary concepts and conventions that we will use and follow in the later part of the article.

Definition 1 An *edge-weighted graph* G is a collection of nodes V, edges E, and a mapping between edge-set E and weights.

For a function $\boldsymbol{W_{edge}(e)}$ that returns the weight of edge $\boldsymbol{e}$, the weight of a subgraph $\boldsymbol{g}$ is defined as:

$$W(g) = \frac{\sum_{e_i \in g} W_{edge}(e_i)}{|g|} \tag{7}$$

where each e_i is an edge of g and $|g|$ denotes the number of edges in subgraph g. In other words, the weight of a subgraph g is the average of the weights of the edges of g. One of the major benefits of this subgraph weighting technique is in WFSM, it will not show biases towards large subgraphs. This weighting technique has been used in [43] which we will use as a baseline for performance evaluation in Section 6.

Support set ρ of a subgraph g in a graph database GDB is the set of all input graphs of GDB where g occurs as a subgraph. If $V(x)$ and $E(x)$ denote nodes and edges of any arbitrary graph x, we can formally define support set of a subgraph g in a graph database GDB as follows:

$$\rho(g, GDB) = \{G | G \in GDB \text{ and } V(g) \subseteq V(G) \text{ and } E(g) \subseteq E(G)\} \tag{8}$$

The cardinality of the support set of a subgraph is called its support. Using subgraph support and weight, weighted support $WSUP$ is calculated. $WSUP$ calculation is discussed later. Given a graph database GDB of weighted graphs and a minimum weighted support threshold τ, a subgraph g is said to be **weighted frequent** if $WSUP(g, GDB) \geq \tau$.

Both the algorithms, WFSM-MaxPWS and DewgSpan, start with an empty subgraph and gradually extend it following a depth-first traversal approach similar to existing substructure mining approaches. Each candidate subgraph is represented by DFS code, a sequential collection of tuples. Here, each tuple represents an edge. Each node of a discovered subgraph has a discovery time. The edge's incident nodes' discovery times are used to present edges using the DFS code tuple. Let us consider an edge from a node labeled u having DFS discovery time dis_u to a node labeled v having DFS discovery time dis_v. If the edge in consideration has label l, it will be represented as a DFS code tuple $t = < dis_u, dis_v, u, v, l >$. In this article, we have used the terms subgraph (represented as g) and DFScode (represented by C) interchangeably. However, in WFSM-MaxPWS, we have extended the DFS code tuple representation for better search space pruning as described in Section 4.

Similar to gSpan, we prune all the DFS codes not associated with the minimum DFS code according to the canonical ordering. The canonical ordering also varies for WFSM-MaxPWS due to the change in DFS tuple representation. For each candidate subgraph with minimum DFS code, we calculate its weighted support($WSUP$) and find if it is a weighted frequent subgraph. We also calculate the MaxPWS value, which is an upper bound of weighted support; that is, no extension of a subgraph can have $WSUP$ value greater than its MaxPWS value. So, we prune candidate subgraphs with MaxPWS value less than the weighted support threshold τ.

4 WFSM with static edge weight

This section describes our algorithm WFSM-MaxPWS for weighted frequent subgraph mining from a weighted graph database with static edge weight. We first show our proposed canonical ordering for graphs with static edge weight and then discuss the MaxPWS value calculation and the algorithm in detail. We have also presented a simulation of the algorithm on a sample graph database.

In a static edge-weighted graph, the weight of an edge only depends on the labels of the edge and associated vertices.

Figure 3 shows a sample graph database $sGDB$ with static edge weight. We have avoided the edge label and node id in Fig. 3 for simplicity.

In the static edge weight setting, the weighted support of a subgraph is calculated by multiplying its weight by its support. So, weighted support, $WSUP$ of g in database sGDB with static edge weight can be defined as:

$$WSUP(g, sGDB) = W(g) \times |\rho(g, sGDB)| \quad (9)$$

4.1 WFSM-MaxPWS Canonical Ordering of Subgraph

For static edge-weighted graphs, we add weight-property in the extended edge-tuple representation used in gSpan. Consequently, the new extended tuple for an edge (u, v) is of the following form:

$$\langle dis_u, dis_v, L(u), L(v), L(u, v), W(u, v)\rangle.$$

Here, $W(u, v)$ is the weight of edge (u, v). To give a rank of a DFScode of the subgraph, the WFSM-MaxPWS canonical order gives the highest priority to (dis_u, dis_v) as in gSpan. The second highest priority is given to edge weight $W(u, v)$. The higher the weight, the smaller the tuple for the same discovery times. The third priority is given to a lexicographic comparison of node and edge label trio. To elaborate, let

$t_1 = \langle v_i, v_j, L(v_i), L(v_j), L(v_i, v_j), W(v_i, v_j)\rangle$; and
$t_2 = \langle v_x, v_y, L(v_x), L(v_y), L(v_x, v_y), W(v_x, v_y)\rangle$.

Then, $t_1 < t_2$ if and only if

1. $(v_i, v_j) <_e (v_x, v_y)$; or
2. $(v_i, v_j)=(v_x, v_y)$ and $W(v_i, v_j) > W(v_x, v_y)$; or
3. $(v_i, v_j)=(v_x, v_y)$ and $W(v_i, v_j)=(v_x, v_y)$ and $\langle L(v_i), L(v_j), L(v_i, v_j)\rangle <_l \langle L(v_x), L(v_y), L(v_x, v_y)\rangle$.

Here, $<_e$ is an ordering on edge, and $<_l$ is an ordering on the vertex and edge labels. $<_l$ follows lexicographic order, so further discussion is unnecessary. The edge order ($<_e$) rule is derived from *rightmost path extension* sequence introduced in [18]. The edge which is extended earlier is smaller. Let $e_{ij} = (v_i, v_j)$ and $e_{xy} = (v_x, v_y)$. We say $e_{ij} <_e e_{xy}$ if and only if:

(a) **[Case 1: e_{ij} Forward Edge, e_{xy} Forward Edge]**
Condition: (i) $j < y$ or (ii) $j = y$ and $i > x$ *Forward extension to a node with lower discovery time is considered smaller. But, if both endpoint nodes have the same discovery time, forward extension from a node with a higher discovery time is considered smaller.*

(b) **[Case 2: e_{ij} Backward Edge, e_{xy} Backward Edge]**
Condition: (i) $i < x$ or (ii) $i = x$ and $j < y$
Backward extension from a node with smaller discovery time is considered smaller. If both edges originate from a node with the same discovery time, then extension to a node closer to the root along the rightmost path is considered smaller.

(c) **[Case 3: e_{ij} Forward Edge, e_{xy} Backward Edge]**
Condition: $j \leq x$
A forward edge to a node with a smaller discovery time is smaller than a backward edge from that node or any node with a higher discovery time.

(d) **[Case 4: e_{ij} Backward Edge, e_{xy} Forward Edge]**
Condition: $i < y$
A backward edge from a node with lower discovery time is smaller than a forward edge to any later node.

Figure 7 shows a comparison between the canonical representations in gSpan [18] and our **WFSM-MaxPWS**. As gSpan puts an edge ordering during the mining time of endpoint nodes and compares the (node labels-edge label)-trio in lexicographic order, tuple t_{11} becomes the smallest for the graph in the figure. On the other hand, for our WFSM-MaxPWS, after performing edge ordering during the node mining time, the second importance is put on edge weight. Hence, the *higher the weight, the smaller the tuple*. As the first tuple of any DFScode has node discovery pair (0, 1), the highest weighted edge would be the smallest. So, WFSM-MaxPWS would consider tuple $\langle 0, 1, b, d, 0.9\rangle$ as the smallest. DFScode with other edges as the first tuple would not be canonical.

Lemma 1 *No tuple can have a higher weight than the weight of the first tuple in a canonical WFSM-MaxPWS DFScode.*

Proof (By induction on edge count m) The first tuple is the only one for the base case (when $m = 1$). Hence, there is no other tuple with a heavier weight.

Then, for the inductive step, let us assume that the hypothesis is true (i.e., DFScode C_x for $1 < m \leq x$, tuples with a first tuple as t_{x0}=⟨0,1, L(v_1), L(v_2), L(v_1, v_2), W(v_1, v_2)⟩). The hypothesis holds if we extend the subgraph with a lower or equal weighted tuple as the first tuple. However, if we extend with a higher weighted tuplet_{xk} = ⟨*, *, L(u_1), L(u_2), L(u_1, u_2), W(u_1), u_2)⟩, then there exists a new possible DFScode C_y having t_{y0} = ⟨0, 1, L(u_1), L(u_2), L(u_1, u_2), W(u_1, u_2)⟩ as first tuple. $t_{x0} > t_{y0}$ implies that $Rank(C_y) < Rank(\{C_x, t_{xk}\})$. So, $\{C_x, t_{xk}\}$ cannot be canonical. Thus, we cannot extend a subgraph with a higher-weighted tuple while preserving canonicity.

4.2 MaxPWS pruning technique

We divide the entire graph database into multiple partitions $p_1, p_2, p_3, \ldots, p_x, \ldots, p_M$. Partition p_1 is the set of graphs in the database with the minimum number of edges; partition

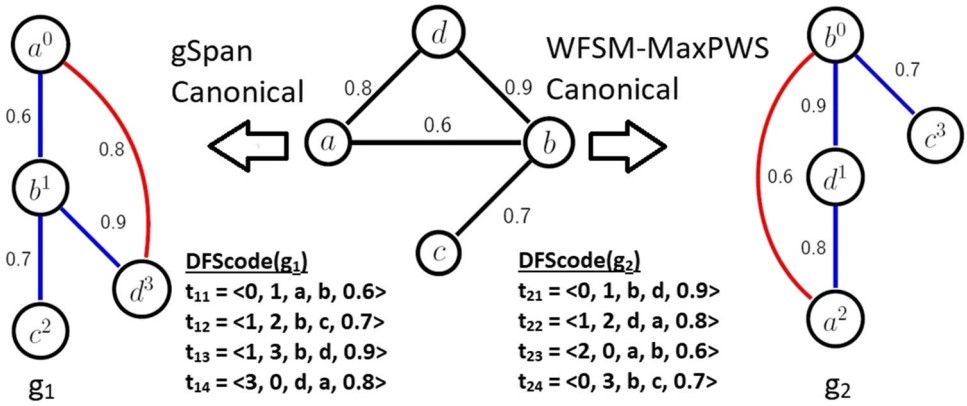


Fig. 7 Canonical Code Comparison with gSpan(without edge-label)

p_M is the collection of graphs with the maximum number of edges. Graphs in the same partition have the same number of edges. Each graph in partition p_x contains at least one more edge than any graphs in partition p_{x-1} and contains at least one fewer edge than graphs in partition p_{x+1}. We called this partition-collection *Edge-Class*.

Definition 2 The Edge Class of a graph database is a collection of partitions on that database where each partition is a set of graphs having an equal number of edges.

There are three partitions for the sample graph database in Fig. 3. Edge Class EC can be represented as follows:

$$EC = \{p_1 = \{G_1\}_4;\ p_2 = \{G_3, G_4\}_6;\ p_3 = \{G_2\}_7\}.$$

For any subgraph of consideration, we calculate *Occurrence-List*, which is a collection of subsets from each Edge-Class partition entry where each member graph of the collection is a superset for that subgraph.

Definition 3 Let g be a subgraph. Then, its occurrence list $OL(g)$ is defined as $OL(g) = \{q_1, q_2, q_3, ...\}$ where $\forall q_i \in OL(g)[q_i \subseteq p_i$ and g is a subgraph of each member of $q_i]$. We call q_i an **O**ccurrence **L**ist **M**ember (**OLM**).

For example, for the database in Fig. 3, the occurrence list for subgraph g_x shown in Fig. 8 is $OL(g_x) = \{q_1 = \{G_1\}_4;\ q_2 = \{G_3\}_6;\ q_3 = \{G_2\}_7\}$.
We can calculate the *possible occurrence list* (*pol*) and the *maximum possible frequency* (*mpf*) for a subgraph based on its occurrence list for extension of that subgraph up to a different edge count.

The possible occurrence list contains a potential support set if we extend a subgraph to different edge counts.

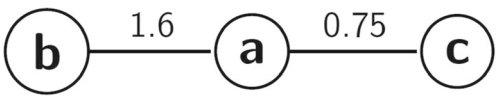


Fig. 8 Sample Subgraph g_x

The cardinality of that set is the maximum possible edge frequency for the corresponding edge count of that subgraph. For instance, from the occurrence list of g_x, if we extend the subgraph g_x up to 4 edges, then the corresponding *pol* of the extended subgraph $g_{x\to4}$ is still $pol(g_{x\to4}) = \{\{G_1\}_4; \{G_3\}_6; \{G_2\}_7\}$. Similarly, the maximum possible frequency of $g_{x\to4}$ becomes $mpf(g_{x\to4}) = 3$. However, if we extend g_x beyond 4 edges, graph G_1 can no longer be a part of its occurrence list because it contains only four edges in total. For extension up to six edges,

$$pol(g_{x\to6}) = \{\{G_3\}_6;\ \{G_2\}_7\} \text{ and } mpf(g_{x\to6}) = 2.$$

Similarly,

$$pol(g_{x\to7}) = \{\{G_2\}_7\} \text{ and } mpf(g_{x\to7}) = 1.$$

Now, the heaviest possible extension of g_x up to 4, 6 and 7 edges can contain 2, 4 and 5 more MaxW-weighted edges, respectively. For the database in Fig. 3, it is 2.0 (edge b-c). According to Lemma 1, this extension can be canonical if and only if the first tuple in the canonical DFScode of g_x has a weight of at least 2.0. But, the canonical DFScode of g_x is $\{\langle 0, 1, a, b, 1.6\rangle, \langle 0, 2, a, c, 0.75\rangle\}$. For simplicity, we have avoided edge labels here. Thus, to be canonical, any extended subgraph of g_x cannot have an edge with a weight heavier than 1.6.

In general, to preserve canonicity, the heaviest possible extension of any subgraph is its **F**irst **T**uple **W**eight (**FTW**). With this concept, for any subgraph g, we can calculate possible weighted support (PWS) after extending up to m-edges through the following equation:

$$PWS(g_{\to m}) = \frac{|g| \times W(g) + (m - |g|) \times FTW}{m} \times mpf(g_{\to m}) \qquad (10)$$

where $|g|$ is the number of edges in g. In the PWS calculation, subgraph g is assumed to be extended up to m edges, with

each new edge having the FTW weight to ensure that this imaginary extended supergraph has the maximum possible weight with respect to the graph database (because extension with the heavier weighted edge would not be canonical). We calculate MaxPWS by taking the maximum among all possible PWS values for g. If MaxPWS fails to satisfy the minimum weighted support threshold τ, we can safely prune g. Otherwise, we need to extend g because its extended subgraphs have the potential to be weighted-frequent.

For example, consider the subgraph g_x. Its $W(g_x) = \frac{1.6+0.75}{2} = 1.175$. For graph database in Fig. 3, PWS values are as follows:

- $PWS(g_{x\to 4}) = \frac{2\times 1.175+(4-2)\times 1.6}{4} \times 3 = 1.3875 \times 3 = 4.1625$
- $PWS(g_{x\to 6}) = \frac{2\times 1.175+(6-2)\times 1.6}{6} \times 2 \approx 1.4583 \times 2 = 2.9166$
- $PWS(g_{x\to 7}) = \frac{2\times 1.175+(7-2)\times 1.6}{7} \times 1 \approx 1.4786 \times 1 = 1.4786$

Hence, $MaxPWS(g_x) = 4.1625$. On the other hand, the MaxW measure for subgraph g_x is $2 \times 3 = 6$ (because $MaxW$=2 and frequency=3), which is greater than MaxPWS.

If we do not use the modified canonical ordering for weighted graphs as proposed in Section 4.1, then the PWS calculation would have to use MaxW (instead of FTW as shown in (11)):

$$PWS(g_{\to m}) = \frac{|g| \times W(g) + (m - |g|) \times MaxW}{m} \times mpf(g_{\to m}) \quad (11)$$

We call this variant of WFSM-MaxPWS as **MaxPWS-gSpan algorithm** and will use this in performance evaluation to illustrate the benefit of our proposed canonical ordering.

Lemma 2 *MaxPWS-measure is anti-monotonic.*

Proof (By contradiction) Suppose MaxPWS is not anti-monotonic. Then, there exists an extended subgraph g_x of g, for which
$MaxPWS(g_x) \geq \tau$ even though $MaxPWS(g) < \tau$. If $|g_x| - |g| = k$, then we can say,

$$W(g_x) \times |g_x| \leq W(g) \times |g| + k \times FTW \quad (12)$$

If MaxPWS of g_x occurs for m edge extension, then-
$PWS(g_{x\to m}) > PWS(g_{\to m})$

$$or, \frac{|g_x| \times W(g_x) + (m - |g_x|) \times FTW}{m} \times mpf(g_{x\to m}) > \frac{|g| \times W(g) + (m - |g|) \times FTW}{m} \times mpf(g_{\to m}) \quad (13)$$

using (10).
Due to the downward closure property of frequency, $mpf(g_x)$ must be less than or equal to $mpf(g)$. So, we can remove the mpf term from both sides.

$$or, |g_x| \times W(g_x) + (m - |g_x|) \times FTW > |g| \times W(g) + (m - |g|) \times FTW$$

$$or, W(g) \times |g| + k \times FTW + (m - |g| - k) \times FTW > |g| \times W(g) + (m - |g|) \times FTW$$

using [(12) and $|g_x| = |g| + k$ relation]

$$or, k \times FTW + (m - |g| - k) \times FTW > (m - |g|) \times FTW$$

$$or, (m - |g|) \times FTW > (m - |g|) \times FTW$$

which is impossible. Thus, $MaxPWS(g_x)$ cannot be greater than $MaxPWS(g)$. So, MaxPWS-measure is anti-monotonic.

From Lemmas 1 and 2, we can reach to following two corollaries:

Corollary 1 *If $MaxPWS(g) < \tau$, then g has no potential weighted frequent extension.*

Corollary 2 *Due to MaxPWS-measure $\leq$ MaxW-measure, the MaxPWS pruning technique prunes more unnecessary patterns.*

4.3 The WFSM-MaxPWS algorithm

Before applying the WFSM-MaxPWS algorithm, we must divide the entire graph database into edge-wise partitions, i.e., *Edge Classes*. We partition the graph database using Algorithm 1. This algorithm takes graph database D as a parameter and returns *Edge Class*: EC, maximum edge count: $maxE$ and minimum edge count: $minE$.

Algorithm 1 Edge Class Assignment

```
1: procedure ASSIGN_EDGE_CLASS(D)
2:     maxE ← 0
3:     minE ← a large number
4:     for each G_i ∈ D do
5:         m_i = |G_i|  [Here, |G_i| stands for edge count of G_i]
6:         if m_i < minE then minE = m_i
7:         if m_i > maxE then maxE = m_i
8:         EC[m_i] = EC[m_i] ∪ gid(G_i)
       [gid gives id of its parameter graph]
9:     return EC, maxE, minE
```

Pseudocode for WFSM-MaxPWS is shown in Algorithm 2. Here, the WFSM-MaxPWS algorithm takes (i) the

canonical DFScode of a graph C, (ii) graph database D, (iii) weighted support threshold τ, and (iv) occurrence list OL_C of C as input parameters. With the initial call, C=$\emptyset$ and OL_C=EC (edge class). Then, WFSM-MaxPWS puts all frequent weighted subgraphs in a set called $result$. The function in Algorithm 3 named "GET_ALL_EXTENSIONS" on line 2 enumerates all possible extensions on the rightmost path of the given DFScode and returns a vector of those extensions as new edge tuples and their corresponding occurrence list $OLvec$. Each entry in $OLvec$ is then checked for canonicity (lines 4 & 5, Algorithm 4). The support of code C' is updated while iterating through each entry in OL (line 9). Simultaneously, possible weighted support from the highest to the lowest $OLM(q_i)$ is calculated to find $MaxPWS$ (lines 8-11). The actual weighted support $wsup$ of the code/subgraph is calculated on line 11. If it satisfies the weighted support threshold condition (line 13), then it is enlisted as a frequent weighted subgraph (line 14) and sent for further extension (line 15). Otherwise, if $MaxPWS$ satisfies the weighted support threshold condition, though it will not be enlisted as a frequent weighted subgraph, then it will be sent to the WFSM-MaxPWS algorithm for further extension (lines 16 and 17) as its extended graph still has a chance to be frequent weighted subgraph.

Algorithm 2 $WFSM_MaxPWS$

1: **procedure** $WFSM_MaxPWS(C, D, \tau, OL_C)$
2: $OLvec = GET_ALL_EXTENSIONS(C, OL_C, D)$
3: **for each** $(t, OL_t) \in OLvec$ **do**
4: $C' = C \cup t$
5: **if** $IS_CANONICAL(C')$ = false **then** continue.
6: Set $sup_{C'}$=0 and $MaxPWS$=0
7: **for each** $q_i \in OL_t$ in reverse order (last to first) **do**
8: m_i = edge count of q_i
9: $sup_{C'} = sup_{C'} + |q_i|$ //cardinality of q_i
10: $PWS_i = \frac{|C'| \times W(C') + (m_i - |C'|) \times FTW}{m_i} \times sup_{C'}$
11: **if** $MaxPWS < PWS_i$ **then** $MaxPWS = PWS_i$
12: $wsup_{C'} = W(C') \times sup_{C'}$
13: **if** $wsup_{C'} \geq \tau$ **then**
14: $result = result \cup C'$
15: $WFSM_MaxPWS(C', D, \tau, OL_t)$
16: **else if** $MaxPWS \geq \tau$ **then**
17: $WFSM_MaxPWS(C', D, \tau, OL_t)$

A subgraph may occur in several positions or several ways in a large graph. To find all possible extensions, we have first to find all possible occurrences of a subgraph in each graph of its occurrence list and look for possible edge extensions. This requires subgraph isomorphism checking. In Algorithm 3, the right-most-path extension of given DFScode C for each graph in the occurrence list is performed. Right-most-path R and right-most-node u_r are first recored(lines 2 and 3). If C is $\emptyset$ (initial call), all edges are taken individually from occurrence list OL_C(*Edge Class* in this case) and are taken as forward edges with discovery time 0 and 1(in both directions). Then, this tuple and its occurrence list OL_t are added in $OLvec$ (lines 7-11). If C is not empty, all subgraph isomorphisms for C in each graph in its occurrence list are considered for edge extensions. This is done by subgraph isomorphism algorithm as explained in [51]. It returns the list of all possible isomorphic existence of subgraph (C in this case) in a given graph (G in the algorithm). For each subgraph isomorphism, both backward edge extension from the rightmost node(lines 15-19) and forward edge extension from the rightmost path(lines 20-24) is considered. The extension tuple and its corresponding occurrence list are recorded in $OLvec$.

Algorithm 3 GET_ALL_EXTENSIONS

1: **procedure** GET_ALL_EXTENSIONS(C, OL_C, D)
2: $R \leftarrow$ nodes on rightmost path of C
3: $u_r \leftarrow$ rightmost child of C
4: $OLvec \leftarrow \emptyset$
5: **for each** $q_i \in OL_C$ **do**
6: **for each** $G_j \in q_i$ **do**
7: **if** $C = \emptyset$ **then**
8: **for each** distinct $[L(x), L(y), L(x, y)] \in G_j$ **do**
9: $t = [0, 1, L(x), L(y), L(x, y), W(x, y)]$
10: Add pair (t, OL_t) in $OLvec$ if not added before
11: Add or modify entry $OL_t(q_i)$: $OL_t(q_i) = OL_t(q_i) \cup G_j$
12: **else**
13: $\Phi_i = SUBGRAPHISOMORPHISMS(C, G_j)$
14: **for each** isomorphism $\phi \in \Phi_i$ **do**
//Backward Edge Extension from rightmost child u_r
15: **for each** $x \in N_{G_j}(\phi(u_r))$ *such that* $\exists v \leftarrow \phi^{-1}(x)$ **do**
16: **if** $v \in R$ and $(u_r, v) \notin G(C)$ **then**
17: $t = [u_r, v, L(u_r), L(v), L(u_r, v), W(u_r, v)]$
18: Add pair (t, OL_t) in $OLvec$ if not added before
19: Add or modify entry $OL_t(q_i)$: $OL_t(q_i) = OL_t(q_i) \cup G_j$
//Forward edge extension from rightmost path R
20: **for each** $u \in R$ **do**
21: **for each** $x \in N_{G_j}(\phi(u))$ and $\nexists \phi^{-1}(x)$ **do**
22: $t = [u, u_r + 1, L(\phi(u)), L(x), L(\phi(u), x), W(\phi(u), x)]$
23: Add pair (t, OL_t) in $OLvec$ if not added before
24: Add or modify entry $OL_t(q_i)$: $OL_t(q_i) = OL_t(q_i) \cup G_j$
25: **return** $OLvec$

To check whether a DFScode is canonical or not, we first generate the graph from the code. Then, we progressively iteratively reconstruct the canonical code starting from empty code and choosing the least rightmost path extension. If, at any step, the partial canonical code is smaller than the given code, then the given code cannot be canonical. Canonicality checking for this work is done by Algorithm 4.

Algorithm 4 IS_CANONICAL

```
1: procedure IS_CANONICAL(C = {t1, t2, t3, ..., tk})
2:     C* ← ∅
3:     DC* = {G(C)}
4:     OLC* = gid(G(C))
5:     for i = 1 to K do
6:         OLvec = GET_ALL_EXTENSIONS(C*, OLC*, DC*)
7:         (si, OLsi) ← min(OLvec)
8:         if si < ti then
9:             return false
10:        C* = C* ∪ si
11:    return true
```

In order to check whether $C = \{t_1, t_2, t_3, ..., t_k\}$ canonical or not, we start with $C^* = \emptyset$(line 2). We extend C^* in a graph database comprising only graph $G(C)$(lines 3 and 4). Then, we extend C^* k times, and at each extension step, we choose the least right-most extension from all possible extensions(lines 5-7). To decide whether C is canonical or not, we compare t_i of C and choose the least extension s_i in the ith step. If s_i is smaller than t_i, C cannot be canonical(lines 8 and 9).Otherwise s_i is added to C^*(line 10). After k-extension, C^* will be the same as C(if not returned before). C will be canonical. So, the algorithm returns true.

4.4 WFSM-MaxPWS simulation

This section shows a partial simulation of the WFSM-MaxPWS algorithm on the sample graph database of Fig. 3 for minimum weighted support threshold, $\tau = 3.0$. The maximum weighted edge of this graph is $b - c$ which has a weight value of 2.0. So, $MaxW = 2.0$. Initial call with $C_0 = \emptyset$ will return eight possible extensions as shown in Fig. 9.

Here, subgraphs with dashed red edges are pruned due to being non-canonical and subgraphs with solid red edges are pruned because $MaxPWS < \tau$. For instance, we see that $C_1 = \{[0, 1, a, a, 1.9]\}$ is canonical but its occurrence list is: $OL(C_1) = \{\{G_4\}_6\}$. So, $F(C_1) = 1$, $W(C_1) = 1.9$ and $wsup(C_1) = 1.9 \times 1 = 1.0 < \tau \rightarrow$ so, it is not weighted

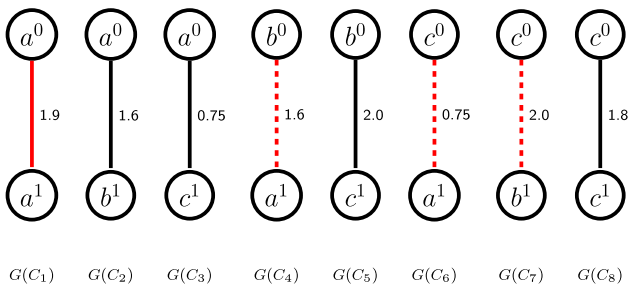


Fig. 9 C_0 Extensions

frequent subgraph. And, $PWS_1 = \frac{1\times1.9+(6-1)\times1.9}{6} \times 1 = 1.9$ So, $MaxPWS = PWS_1 = 1.9 < \tau$ and so will be pruned. The point to be noted here is that if a DFScode's weighted support fails to satisfy the weighted support threshold, but its MaxPWS satisfies, it will be further extended to discover any potential weighted frequent extended subgraph.

On the other hand $C_4 = \{[0, 1, b, a, 1.6]\}$ is pruned for not being canonical. Its isomorphic canonical code is $\{[0, 1, a, b, 1.6]\}$ which is represented by DFScode C_2. Subgraphs with black-gray edges as C_2, C_3, C_5, C_8 are weighted frequent and so will be further extended. Details of for C_0 extension are shown in Table 1.

According to depth-first order, let us consider the extension of C_2 first, shown in Fig. 10 and Table 2. Out of these two extensions, only C_{10} passes the threshold test and is further extended as C_{11} and C_{12} in Fig. 11.

Here, we see that $C_{11} = \{[0, 1, a, b, 1.6], [0, 2, a, c, 0.75], [2, 3, c, c, 1.8]\}$ is non-canonical according to Lemma 1 but $C_{12} = \{[0, 1, a, b, 1.6], [0, 2, a, c, 0.75], [0, 3, a, c, 0.75]\}$ is canonical, and its occurrence list is: $OL_{12} = \{\{G_3\}_6, \{G_2\}_7\}$. Thus, $F(C_{12}) = 2$, $W(C_{12}) = \frac{1.6+0.7.5+0.75}{3} = 1.033$ and $wsup(C_{12}) = 1.033 \times 2 = 2.066 < \tau$; so it is not weighted frequent. For PWS upto 6-edge extension is $PWS_1 = \frac{3\times1.033+(6-3)\times1.6}{6} \times 2 = 2.633$ and up to 7-edge extension, $PWS_2 = \frac{3\times1.033+(7-3)\times1.6}{7} \times 1 = 1.357$. So, $MaxPWS = PWS_1 = 2.6333$ which is also less than the minimum weighted support threshold, $\tau(3.0)$. So, we can safely prune C_{12}. Point to be noted here: if we had followed the $MaxW$ pruning technique, potential weighted support would have been $2.0(MaxW) \times 2(frequency) = 4.0 \geq \tau$. Thus, C_{12} would have been further extended only to generate more unnecessary patterns which will eventually all be pruned. Then, the extension of C_3, C_5 and C_7 will have been completed in a similar way. After completion, we found 14 weighted frequent subgraphs. To get these, WFSM-MaxPWS extends 18 candidate patterns. The MaxW pruning approach will also find 14 weighted frequent subgraphs, but that approach requires extending more candidate patterns, 25 to be specific.

Table 3 shows a comparative chart between the WFSM-MaxPWS algorithm and basic gSpan with the MaxW pruning technique. In the table, $\#WFSPs$, $\#SE$, $\#GE$ and EP stand for the number of weighted frequent subgraph patterns, number of subgraphs extended due to MaxPWS satisfying weighted support threshold τ, the total number of generated subgraphs and extension precision respectively. Extension precision in the table is calculated by (14):

$$EP = \frac{\#\text{WFSPs}}{\#\text{SE}} \tag{14}$$

Table 1 C_0 extensions

DFScode	Canonical	Occurrence List	wsup	WFS	MaxPWS	Extend
$C_1 = \{[0, 1, a, a, 1.9]\}$	✓	$OL_1 = \{\{G_4\}_6\}$	1.9	×	1.9	×
$C_2 = \{[0, 1, a, b, 1.6]\}$	✓	$OL_2 = \{\{G_1\}_4, \{G_3\}_6, \{G_2\}_7\}$	4.8	✓	–	✓
$C_3 = \{[0, 1, a, c, 0.75]\}$	✓	$OL_1 = \{\{G_1\}_4, \{G_3, G_4\}_6, \{G_2\}_7\}$	3.0	✓	–	✓
$C_4 = \{[0, 1, b, a, 1.6]\}$	×	–	–	–	–	×
$C_5 = \{[0, 1, b, c, 2.0]\}$	✓	$OL_5 = \{\{G_1\}_4, \{G_2\}_7\}$	4.0	✓	–	✓
$C_6 = \{[0, 1, c, a, 0.75]\}$	×	–	–	–	–	×
$C_7 = \{[0, 1, c, b, 2.0]\}$	×	–	–	–	–	×
$C_8 = \{[0, 1, c, c, 1.8]\}$	✓	$OL_8 = \{\{G_3, G_4\}_6, \{G_2\}_7\}$	5.4	✓	–	✓

5 WFSM with dynamic edge weight

In this section, we present the extended WFSM-MaxPWS framework incorporating dynamic edge weight. We have discussed the weight distribution table defined and the MaxPWS value used in the algorithm. We also present a simulation of the *DewgSpan* algorithm after the algorithm details.

5.1 Preliminaries

In a dynamic edge-weighted graph, the weight of two edges with the same label set can vary. Figure 5 shows a sample graph database with dynamic edge weight where inside each node there are id and label of the node separated by a vertical bar ("|"). Each edge also has a label along with its weight.

In static edge-weighted WFSM, (7) can be directly used to calculate the weight of a subgraph in an input graph (graph of the database) as all isomorphisms of a subgraph g in that input graph will have the same weight. However, we face a new choice dilemma about edge weight and subgraph weight in a dynamic edge weight setting. For example, while considering subgraph $< (_, _, a, b, y) >$ in input graph G_1 of Fig. 5, what will be the value of subgraph weight? 0.2 / 0.3 or 0.5? It may be the case that if we consider 0.5 as edge weight, then the pattern, in turn, will be weighted frequently in the entire graph database, and if we consider 0.2 instead, it may fail the threshold check. So, what is the most suitable weight to choose remains debated and depends highly on application. However, in this article, following the footsteps of [35], we pick the isomorphism with maximal weight but to avoid biases towards large subgraphs, calculate subgraph weight according to [43]. We call this weighting technique $MaxWeight$.

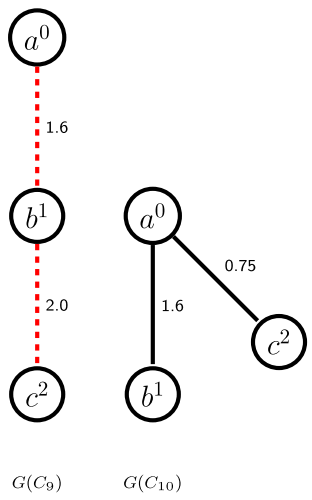


Fig. 10 C_2 Extensions

Definition 4 The average edge weight of the maximal weighted subgraph in an input graph is called $MaxWeight$ of the corresponding subgraph in the dynamic edge weight setting.,

Example 1 Let us calculate the MaxWeight of pattern $g_k = \{(0, 1, a, b, y), (1, 2, b, b, z)\}$ of Fig. 6 in the graph G_1 of the database in Fig. 5.
In graph G_1, subgraph g_k has three isomorphisms which are shown in Table 4
So, the maximum total weight of g_k in G_1 is for ϕ_2, which is $0.5+0.9 = 1.4$. As g_k has two edges, $|g_k| = 2$. So, maximum weight of g_k in G_1, $MaxWeight(g_k, G_1) = 1.4/2 = 0.7$.

Formally, we can define the MaxWeight of an arbitrary subgraph g in graph G as

$$MaxWeight(g, G) = \frac{\max_{\phi \in \Phi(g,G)} TW(\phi)}{|g|} \tag{15}$$

Here, function $TW(\phi)$ returns the edge weight summation of a particular isomorphism ϕ of g in G.

The weight of a subgraph in dynamic edge weight setting, i.e., $MaxWeight$, differs from graph to graph, and so, weighted support is calculated by taking the summation of $MaxWeight$ from all the graphs in its support set. So, weighted support, $WSUP$ of g in database dGDB with

Table 2 C_2 extensions

DFScode	Canonical	Occurrence List	wsup	WFS	MaxPWS	Extend
C_9 = {[0, 1, a, b, 1.6], [1, 2, b, c, 2.0]}	×	–	–	–	–	×
C_{10} = {[0, 1, a, b, 1.6], [0, 2, a, c, 0.75]}	✓	OL_{10} ={{G_1}$_4$, {G_3}$_6$, {G_2}$_7$}	3.525	✓	–	✓

dynamic edge weight can be defined as:

$$WSUP(g, dGDB) = \sum_{G \in \rho(g, dGDB)} MaxWeight(g, G) \tag{16}$$

Example 2 Weighted support calculation of g_k of Example 1.

We observed from Fig. 5 that g_k occurs in all four graphs in the database. So, $\rho(g_k, dGDB) = \{G_1, G_2, G_3, G_4\}$.

With (15), we get max weight:

- $MaxWeight(g_k, G_1) = 0.7$
- $MaxWeight(g_k, G_2) = 0.65$
- $MaxWeight(g_k, G_3) = 0.75$
- $MaxWeight(g_k, G_4) = 0.65$

We get $WSUP(g_k, dGDB) = 0.6166+0.625+0.6375+0.5333 = 2.75$ by using (16).

5.2 Weight distribution table

We propose to save a summary of weight distribution as a table in memory. We call this table **WDT**: **W**eight **D**istribution **T**able. Each graph of the database has a column in WDT. Each row of WDT represents a bin that can hold a count of a certain range of weight values meaning each bin has a *binMax* and a *binMin* value. An edge e_{ij} of graph G_i satisfying the condition $binMax(n-1) < weight(e_{ij}) \leq binMax(n)$ will increase the count of cell WDT[n][i] by one. Depending on available memory, the user can adjust the number of bins(rows) and make groups consisting of multiple graphs to lower the required number of columns. This indeed comes with a compromise with performance. If the maximum and minimum weighted edges of a database are $MaxW$ and $MinW$, respectively, then we calculate the span of each bin using (17) where $binCount$ is the number of bins/rows in WDT.

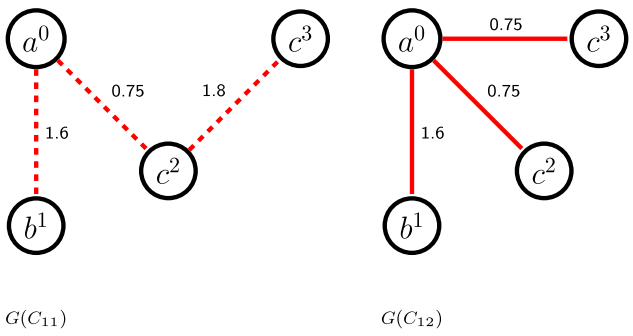


Fig. 11 C_{10} Extensions

$$span = (MaxW - MinW)/binCount \tag{17}$$

And then, using (18), we can get the index i of the bin in which a certain edge with weight x is allocated. Besides, (19) gives the value of $binMax$ of the ith bin of WDT.

$$i = \left\lfloor \frac{x - MinW}{span} \right\rfloor \tag{18}$$

$$binMax(i) = MinW + ((i+1) * span) \tag{19}$$

Example 3 Weight Distribution Table(WDT) of dGDB in Fig. 5 with $binCount = 4$ and no grouping.

Here, MaxW=0.9 and MinW = 0.2; let us first calculate span of bins using (17):
$span = (0.9 - 0.2)/4 = 0.175$ As we have four bins with index 0, 1, 2 and 3, using (19), we can calculate $binMax$ for each bin.

- binMax(0) = 0.2 + (0+1)*0.175 = 0.375
- binMax(1) = 0.2 + (1+1)*0.175 = 0.55
- binMax(2) = 0.2 + (2+1)*0.175 = 0.725
- binMax(3) = 0.2 + (3+1)*0.175 = 0.9

So, in graph G_1, edge (1,2) with weight 0.3 and (2,3) with weight 0.2 will be assigned to bin(0). Similarly, the bin-index for edge(1,4) with a weight of 0.5 will be 1, and for edge(2,4) with a weight of 0.9 will be 3. Complete WDT of $dGDB$ for $binCount = 4$ is shown in Table 5.

Table 3 Simulation summary for sGDB in Fig. 3

Algorithm	#WFSP	#SE	#SG	EP
WFSM-MaxPWS	14	18	90	77.78%
MaxW-gSpan	14	25	98	56.00%

Table 4 Isomorphisms of g_k

Φ	mapping	edge weight β
ϕ_1	{(0, 1)(1, 2), (2, 4)}	[0.3, 0.9]
ϕ_2	{(0, 1)(1, 4), (2, 2)}	[0.5, 0.9]
ϕ_1	{(0, 3)(1, 2), (2, 4)}	[0.2, 0.9]

5.3 Maximum possible weighted support

We propose calculating isomorphism specific Maximum Possible Weight(iso_MPW) for each isomorphism of a subgraph in an original input graph. To calculate iso_MPW for an isomorphism ϕ, we need the WDT index of the input graph, weight summation of ϕ and partial wdt of ϕ, ϕ_wdt. We can represent ϕ as a list of pairs (x, y) where x is a node in the subgraph pattern and y is the corresponding matched node identifier in the input graph in consideration. Partial wdt of ϕ, ϕ_wdt is an array of integers where index represents $WDT\ bin - Index$ and value represents the number of tuples of ϕ falling in that bin.

Example 4 Isomorphisms(Φ) and corresponding ϕ_weight of DFSCode $C_x = \{(0, 1, a, b, y), (1, 2, b, b, z), (2, 0, b, a, y)\}$ in input graph G_4.

Pattern C_k has four isomorphisms in G_4. They are:

- $\Phi[0] = [(0, 2), (1, 1), (2, 3)]$, $\beta(\Phi[0]) = [0.2, 0.6, 0.4]$
- $\Phi[1] = [(0, 2), (1, 3), (2, 1)]$, $\beta(\Phi[1]) = [0.4, 0.6, 0.2]$
- $\Phi[2] = [(0, 4), (1, 3), (2, 5)]$, $\beta(\Phi[2]) = [0.7, 0.6, 0.4]$
- $\Phi[3] = [(0, 4), (1, 5), (2, 3)]$, $\beta(\Phi[3]) = [0.4, 0.6, 0.7]$

For WDT in Example 3, ϕ_wdt of $\phi = \Phi[0]$ or $\Phi[1]$ will be [1, 1, 1, 0] meaning 1 edge from $bin[0]$, 1 edge from $bin[1]$, 1 edge from $bin[2]$ and no edge from $bin[3]$. Similarly, ϕ_wdt of $\Phi[2]$ or $\Phi[3]$ will be [0, 1, 2, 0].

Given all the arguments mentioned above, we can calculate iso_MPW for a certain $\phi = \Phi[i]$ using Algorithm 5.

The input graph-specific Maximum Possible Weight (inpG_MPW) is calculated by taking the maximum among all iso_MPWs of a subgraph in an input graph. If Φ is the set of isomorphisms of DFScode C in input graph G, then

$$inpG_MPW(C, G) = \max_{\phi \in \Phi} iso_MPW(\phi, G) \tag{20}$$

Table 5 WDT of dGDB for binCount = 4

Index	binMax	G_1	G_2	G_3	G_4
0	0.375	2	0	1	1
1	0.550	1	2	1	2
2	0.725	0	1	1	3
3	0.900	1	2	2	1

Algorithm 5 ISO_MPW

```
1: procedure GET_ISO_MPW(index, φ, φ_wdt)
2:     len = length(φ)
3:     weightSum = TW(φ)
4:     iso_MPW = weightSum / length(φ)
5:     for i = binCount − 1 to 0 do
6:         if binMax(i) > iso_MPW then
7:             RemEdge_i = WDT[i][index] - φ_wdt[i]
8:             weightSum+ = binMax(i) × RemEdge
9:             len+ = RemEdge_i
10:            iso_MPW = weightSum / len
11:        else
12:            break
13:    return iso_MPW
```

Example 5 Calculation of inpG_MPW for DFScode, $C_k = \{(0, 1, a, b, y), (1, 2, b, b, z), (2, 0, b, a, y)\}$ in G_4.

For each isomorphism $\phi \in \Phi$ calculated in Example 4, we have to calculate iso_MPW using Algorithm 5. Let us first consider $\phi = \Phi[0] = [(0, 2), (1, 1), (2, 3)]$ for which $\beta = [0.2, 0.6, 0.4]$ and $\phi_wdt = [1, 1, 1, 0]$. Initial $weightSum = TW(\phi) = 1.2$ and length, $len = 3$. So, initial $iso_MPW = \frac{1.2}{3} = 0.4$.

In loop [binCount-1 = 3]:
binMax(3) = 0.9 which is greater than current iso_MPW.
RemEdge = $WDT[3][G_4]$ - $\phi_wdt[3]$ = 1 - 0 = 1
weightSum = weightSum + binMax(3)*RemEdge = 1.2 + 0.9*1 = 2.1
len = len + RemEdge = 3 + 1 = 4
updated $iso_MPW = \frac{2.1}{4} = 0.525$

Similarly, in loop [2]:
binMax(2) = 0.725 > $iso_MPW = 0.525$
RemEdge = $WDT[2][G_4]$ - $\phi_wdt[2]$ = 3 - 1 = 2
weightSum = 2.1 + 0.725*2 = 3.55
len = len + RemEdge = 4 + 2 = 6
updated $iso_MPW = \frac{3.55}{6} \approx 0.592$

loop[1] will not be executed as binMax(1) = 0.550 which is less than latest found $iso_MPW = 0.592$ So, $iso_MPW(\Phi[0], G_4) \approx 0.592$. Similarly,

- $iso_MPW(\Phi[1], G_4) \approx 0.592$
- $iso_MPW(\Phi[2], G_4) \approx 0.665$
- $iso_MPW(\Phi[3], G_4) \approx 0.665$

Now, using (20), we get $inpG_MPW(C_k, G_4) = 0.665$.

Taking the sum of $inpG_MPW$ of a subgraph g(or interchangeably its DFS code C) over its support set will give us **Max**imum **P**ossible **W**eighted **S**upport: **MaxPWS** that any of g's supper-pattern may probably have.

$$MaxPWS(g) = \sum_{G \in \rho(g, dGDB)} inpG_MPW(g, G) \quad (21)$$

By using (21) for $C_k = \{(0, 1, a, b, y), (1, 2, b, b, z), (2, 0, b, a, y)\}$ over its support set, $\rho(C_k, GDB) = \{G_1, G_3, G_4\}$, we can find $MaxPWS(C_k) \approx 1.9316$. And, from Section 3 and (16), we get that the Weighted Support of C_k is 1.7666 using MaxWeight.

5.4 The DewgSpan algorithm

DewgSpan works in a depth-first fashion similar to gSpan [18] and WFSM-MaxPWS [50].

We briefly describe the DewgSpan framework with MaxWeight/AvgWeight as WeightMeasure here.

Step 1: Read the database and prepare weight distribution table WDT as described in Section 5.2.

Step 2: After reading the dataset, an initial call is made with zero length canonical code $C = C_0$, i.e., empty subgraph. Recursive mining is then performed by discovering extended candidates of C using the DewgSpan algorithm.

Step 3: For each input graph G, DewgSpan records all isomorphism ϕ_C of C in G: $\Phi_C(G)$ (Algorithm 6 lines 3-4).

Step 4: For each isomorphism $\phi_C \in \Phi_C(G)$, single edge extension is discovered using "Right Most Path Extension" policy introduced in gSpan. Extended patterns are recorded in groups where each individual group: Φ contains isomorphic instances of a unique extended subgraph g_x for example (Algorithm 6 lines 5-9).

Step 5: Weight of each g_x is calculated using (15) depending on the weight measure choice (MaxWeight /AvgWeight).

Step 6: For each isomorphism $\phi \in \Phi$ of g_x, its partial weight distribution ϕ_wdt is extracted and maximum possible weight for that isomorphism in G, $iso_MPW(\phi, G)$ is calculated using Algorithm 5. Afterward, the maximum possible weight of g_x is calculated using (20) (Algorithm 6 lines 10-14).

Step 7: Maximum Possible Weighted Support: MaxPws of g_x is then derived from the support set of g_x in the entire database by (21). Similarly, Weighted Support: WSUP of g_x is calculated using (16) (Algorithm 6 lines 15-18).

Step 8: If $WSUP(g_x) \geq threshold$, then g_x is recorded in the result set as weighted-frequent and a recursive call is made with canonical code of g_x. Otherwise, if $MaxPWS(g_x) \geq threshold$ then g_x, although not recorded in the result set, a recursive call is made to discover the extended weighted frequent patterns of g_x if any. If MaxPWS fails to satisfy the threshold, g_x is pruned immediately (Algorithm 6 lines 19-22).

Algorithm 6 DewgSpan

```
1: procedure DewgSpan(C, WDT, D)
2:   for each G ∈ D do
3:     for each isomorphism of C in G φ_C do
4:       Φ_C(G) = Φ_C(G) ∪ φ_C
5:     for each φ_C ∈ Φ_C(G) do
6:       S ⟵ Right_Most_Path_Extension(G, φ_C)
7:       UniqueExtension = UniqueExtension ∪ S
8:       for each φ_gx ∈ S do
9:         Φ[g_x].append(φ_gx)
10:    for each phi ∈ Φ[g_x] do
11:      φ_wdt = ExtractPartialWDT(WDT, φ)
12:      iso_MPW(φ) = ISO_MPW(g_x, φ, φ_wdt)
13:      if inpG_MPW[G][g_x] < iso_MPW(φ) then
14:        inpG_MPW[G][g_x] = iso_MPW(φ)
15:  for each g ∈ UniqueExtension do
16:    for each G ∈ ρ(g, D) do
17:      MaxPWS(g, D)+ = inpG_MPW[G][g]
18:      WSUP(g, D)+ = MaxWeight(g, G)
19:    if WSUP(g, D) > threshold then
20:      Result = Result ∪ g
21:    if MaxPWS(g, D) > threshold then
22:      DewgSpan(g, WDT, D)
```

5.5 DewgSpan simulation

This section presents a partial simulation of the sample graph database $dGDB$ shown in Fig. 5 following $MaxWeight$ as $WeightMeasure$. According to *Step 1* of the DewgSpan framework presented in Section 5, first, the database is read, and the weight distribution table is constructed. This simulation considers $bincount$ as 4 and weighted support threshold $\tau = 2.5$. In Table 5, the resulting WDT is shown already. $DewgSpan$ starts recursive mining (*Step 2*), making an initial call with DFScode $C_0 = \{\}$ of the empty subgraph, which will give all 1-edge subgraphs and their isomorphisms for each input graph $G_i \in GDB$ (*Steps 3 and 4*).

Table 6 shows all extended subgraphs of C_0 in input graph G_1. Similarly, all 1-edge extensions of C_0 in $\{G_2, G_3, G_4\}$ are also calculated. For all discovered subgraphs, $MaxWeight$ and $inpG_MPW$ are calculated using (15) and Algorithm 5 with (20) which is shown in Table 7. From Table 7, we can calculate $WSUP$ and $MaxPWS$ of each sub-

Table 6 Right Most Path Extension of C_0 in G_1

Subgraph	DFS code	Isomorphisms Φ	Weight β
C_1	$[<0,1,a,b,y>]$	$[(0,1),(1,2)]$	[0.3]
		$[(0,1),(1,4)]$	[0.5]
		$[(0,3),(1,2)]$	[0.2]
C_2	$[<0,1,b,a,y>]$	$[(0,2),(1,1)]$	[0.3]
		$[(0,4),(1,1)]$	[0.5]
		$[(0,2),(1,3)]$	[0.2]
C_3	$[<0,1,b,b,z>]$	$[(0,4),(1,2)]$	[0.9]
		$[(0,2),(1,4)]$	[0.9]

graph/DFScode by taking the sum of their corresponding $MaxWeight$ and $inpG_MPW$ value, respectively. Table 8 shows the calculated value of $WSUP$ and $maxPWS$ of each discovered subgraph and also shows whether they will be pruned or extended. Pattern C_1 and C_3 are canonical and $WSUP(C_i) \geq \tau$ (here, i = 1 or 3), so they are weighted frequent and will be extended. Pattern C_2 will be pruned without checking the threshold condition as it is not canonical according to the definition introduced in gSpan [18]. And, in the case of Pattern C_4, its $WSUP$ and $MaxPWS$ both fail to satisfy the threshold condition and thus will be pruned.

Next, following the depth-first-traversal approach, $DewgSpan$ will first discover all patterns that can be derived from C_1, and then it will extend C_3. All $len(C_1) + 1$ extended subgraphs of C_1 in $\{G_1, G_2, G_3, G_4\}$ are shown in Table 9. In Table 9, we have includeded all isomorphisms (ϕ) of each extended patterns, their weight vector (w), partial weight distribution table ($\phi_w dt$) followed by their iso_MPW, $MaxWeight$ and $inpG_MPW$ measurements. By taking column-wise summation of $MaxWeight$ and inp_MPW columns grouped by subgraph, we can calculate $WSUP$ and $MaxPWS$, respectively. Table 10 shows the calculated values and their (canonical, weighted frequent, extend) status.

Table 7 MaxWeight and $inpG_MPW$ of all 1-edge subgraphs

Subgraph	DFScode	Graph	MaxWeight	$inpG_MPW$
C_1	$[<0,1,a,b,y>]$	G_1	0.5	0.7
		G_2	0.9	0.9
		G_3	0.8	0.8
		G_3	0.7	0.8
C_2	$[<0,1,b,a,y>]$	G_1	0.5	0.7
		G_2	0.9	0.9
		G_3	0.8	0.8
		G_3	0.7	0.8
C_3	$[<0,1,b,b,z>]$	G_1	0.9	0.9
		G_2	0.8	0.85
		G_3	0.7	0.7667
		G_4	0.6	0.75
C_4	$[<0,1,a,a,x>]$	G_2	0.6	0.7667
		G_4	0.9	0.9

Table 8 WSUP and MaxPWS of C_0 extensions

Subgraph	WSUP	MaxPWS	Canonical?	WFSP?	Extend?
C_1	2.9	3.2	✓	✓	✓
C_2	2.9	3.2	×	—	×
C_3	3.0	3.2667	✓	✓	✓
C_4	1.5	1.6667	✓	×	×

We observed from Table 10 that pattern C_8 will be pruned immediately as it is not canonical and pattern C_5 will be pruned as $WSUP(C_5)$ and $MaxPWS(C_5)$ both fail to satisfy threshold condition. Pattern C_6 will be included in the weighted frequent list and be extended as $WSUP(C_6) \geq \tau$. And, weighted support of pattern C_7, although it fails to satisfy threshold condition, its $MaxPWS$ value is greater than threshold $\tau = 2.5$ which implies it may have weighted frequent super-patterns so C_7 will be further extended. According to the depth-first-traversal approach of $DewgSpan$, C_6 will be extended first, then C_7 and after that, $DewgSpan$ will get back to C_3 and extend in a similar fashion.

6 Performance evaluation

In this section, we show a comparative analysis of performance for our proposed algorithms $WFSM - MaxPWS$ and $DewgSpan$. At first, we have presented the experimental results and analysis of the algorithms and comparison with baseline algorithm $MaxW-gSapn$ devised from [43]. Then, we compared the performances of the algorithms $WFSM - MaxPWS$ and $DewgSpan$ with each other on static weighted graphs as both of the algorithms are compatible with such graphs.

6.1 Experimental settings

We have conducted the experiments of the $WFSM - MaxPWS$ algorithm on a PC with an Intel Core i3-2100 CPU at 3.10GHz and 4 GB RAM running MS Windows 10 operating system, whereas experiments for $DewgSpan$ algorithm has been conducted in a Windows 10 Home (64 bit) machine with 16GB DDR4 RAM, Intel(R) Core(TM) i7-8750H CPU @2.20 GHz. And, experiments for comparing WFSM-MaxPWS and $DewgSpan$ have been performed in a

Table 9 Right Most Path Extension of C_1 in G_1

Subgraph	DFScode	Graph	Isomorphism ϕ	Weight β	ϕ_wdt	iso_MPW	MaxWeight	$inpG_MPW$
C_5	$[<0,1,a,b,y>$	G_1	[(0, 1), (1, 2), (2, 3)]	[0.3, 0.2]	[2, 0, 0, 0]	0.4875	0.25	0.4875
	$<1,2,b,a,y>]$		[(0, 3), (1, 2), (2, 1)]	[0.2, 0.3]	[2, 0, 0, 0]	0.4875		
		G_3	[(0, 1), (1, 2), (2, 4)]	[0.8, 0.8]	[0, 0, 0, 2]	0.8	0.8	0.8
			[(0, 4), (1, 2), (2, 1)]	[0.8, 0.8]	[0, 0, 0, 2]	0.8		
			[(0, 1), (1, 3), (2, 4)]	[0.3, 0.4]	[1, 1, 0, 0]	0.605		
			[(0, 4), (1, 3), (2, 1)]	[0.4, 0.3]	[1, 1, 0, 0]	0.605		
		G_4	[(0, 2), (1, 3), (2, 4)]	[0.4, 0.7]	[0, 1, 1, 0]	0.69	0.55	0.69
			[(0, 4), (1, 3), (2, 2)]	[0.7, 0.4]	[0, 1, 1, 0]	0.69		
C_6	$[<0,1,a,b,y>$	G_1	[(0, 1), (1, 2), (2, 4)]	[0.3, 0.9]	[1, 0, 0, 1]	0.6	0.7	0.7
	$<1,2,b,b,z>]$		[(0, 1), (1, 4), (2, 2)]	[0.5, 0.9]	[0, 1, 0, 1]	0.7		
			[(0, 3), (1, 2), (2, 4)]	[0.2, 0.9]	[1, 0, 0, 1]	0.55		
		G_2	[(0, 4), (1, 1), (2, 2)]	[0.5, 0.8]	[0, 1, 0, 1]	0.73333	0.65	0.73333
			[(0, 3), (1, 2), (2, 1)]	[0.4, 0.8]	[0, 1, 0, 1]	0.70625		
		G_3	[(0, 1), (1, 2), (2, 3)]	[0.8, 0.7]	[0, 0, 1, 1]	0.8	0.75	0.8
			[(0, 1), (1, 3), (2, 2)]	[0.3, 0.7]	[1, 0, 1, 0]	0.7		
			[(0, 4), (1, 2), (2, 3)]	[0.8, 0.7]	[0, 0, 1, 1]	0.8		
			[(0, 4), (1, 3), (2, 2)]	[0.4, 0.7]	[0, 1, 1, 0]	0.725		
		G_4	[(0, 2), (1, 3), (2, 1)]	[0.4, 0.6]	[0, 1, 1, 0]	0.67	0.65	0.73333
			[(0, 2), (1, 3), (2, 5)]	[0.4, 0.6]	[0, 1, 1, 0]	0.67		
			[(0, 2), (1, 1), (2, 3)]	[0.2, 0.6]	[0, 1, 1, 0]	0.63		
			[(0, 4), (1, 3), (2, 1)]	[0.7, 0.6]	[0, 0, 2, 0]	0.73333		
			[(0, 4), (1, 3), (2, 5)]	[0.7, 0.6]	[0, 0, 2, 0]	0.73333		
			[(0, 4), (1, 5), (2, 3)]	[0.4, 0.6]	[0, 1, 1, 0]	0.67		
C_7	$[<0,1,a,b,y>$	G_1	[(0, 1), (1, 4), (2, 2)]	[0.5, 0.3]	[1, 1, 0, 0]	0.56666	0.4	0.56666
	$<0,2,a,b,y>]$		[(0, 1), (1, 2), (2, 4)]	[0.3, 0.5]	[1, 1, 0, 0]	0.56666		
		G_2	[(0, 4), (1, 1), (2, 5)]	[0.5, 0.9]	[0, 1, 0, 1]	0.76666	0.7	0.76666
			[(0, 4), (1, 5), (2, 1)]	[0.9, 0.5]	[0, 1, 0, 1]	0.76666		
		G_3	[(0, 1), (1, 2), (2, 3)]	[0.8, 0.3]	[1, 0, 0, 1]	0.68125	0.6	0.70625
			[(0, 1), (1, 3), (2, 2)]	[0.3, 0.8]	[1, 0, 0, 1]	0.68125		
			[(0, 4), (1, 2), (2, 3)]	[0.8, 0.4]	[0, 1, 0, 1]	0.70625		
			[(0, 4), (1, 3), (2, 2)]	[0.4, 0.8]	[0, 1, 0, 1]	0.70625		
		G_4	[(0, 2), (1, 1), (2, 3)]	[0.2, 0.4]	[1, 1, 0, 0]	0.6125	0.55	0.69
			[(0, 2), (1, 3), (2, 1)]	[0.4, 0.2]	[1, 1, 0, 0]	0.6125		
			[(0, 4), (1, 3), (2, 5)]	[0.7, 0.4]	[0, 1, 1, 0]	0.69		
			[(0, 4), (1, 5), (2, 3)]	[0.4, 0.7]	[0, 1, 1, 0]	0.69		
C_8	$[<0,1,a,b,y>$	G_2	[(0, 3), (1, 2), (2, 4)]	[0.4, 0.6]	[0, 1, 1, 0]	0.7	0.75	0.8
	$<0,2,a,a,x>]$		[(0, 4), (1, 1), (2, 3)]	[0.5, 0.6]	[0, 1, 1, 0]	0.725		
			[(0, 4), (1, 5), (2, 3)]	[0.9, 0.6]	[0, 0, 1, 1]	0.8		
		G_4	[(0, 2), (1, 1), (2, 4)]	[0.2, 0.9]	[1, 0, 0, 1]	0.655	0.8	0.8
			[(0, 2), (1, 3), (2, 4)]	[0.4, 0.9]	[0, 1, 0, 1]	0.695		
			[(0, 4), (1, 3), (2, 2)]	[0.7, 0.9]	[0, 0, 1, 1]	0.8		
			[(0, 4), (1, 5), (2, 2)]	[0.4, 0.9]	[0, 1, 0, 1]	0.695		

Table 10 WSUP and MaxPWS of C_1 extensions

Subgraph	WSUP	MaxPWS	Canonical?	WFSP?	Extend?
C_5	1.6	1.9775	✓	×	×
C_6	2.75	2.9667	✓	✓	✓
C_7	2.25	2.7296	✓	×	✓
C_8	1.55	1.6	×	—	×

Windows 10 Home (64-bit) machine with 8GB DDR4 RAM, Intel(R) Core(TM) i5-8265UH CPU @1.60GHz. All algorithms have been implemented in Python 3.7. The datasets analyzed during the current study are available in the Github[2] repository.

6.2 Performance analysis of WFSM-MaxPWS

We have used three real datasets of cancer screening for this purpose. We have evaluated the performance of our proposed WFSM-MaxPWS algorithm on three real datasets. For a baseline comparison, we have used a weight-adapted version of gSpan MaxW-gSpan that uses the MaxW pruning technique. To illustrate the benefits of our modified canonical ordering for weighted graphs, we have also compared WFSM-MaxPWS with MaxPWS-gSpan where MaxPWS is calculated using (11) instead of (10). We present here a comparison of three criteria: (a) runtime optimization, (b) search-space reduction, and (c) memory requirement efficiency.

6.2.1 Data description

PubChem (National Center for Biotechnology Information, PubChem Compound Database) provides information on the biological activities of small molecules, containing the bioassay records for anti-cancer screen tests with different cancer cell lines. Several datasets were collected from PubChem by Yan [52]. Each dataset belongs to a certain type of cancer screening with the outcome active or inactive. Among those datasets, we have used active labeled **MCF-7**, **P388** and **Yeast** datasets. Figure 12 shows a randomly picked graph from the MCF-7 dataset.

6.2.2 Weight inclusion

The original graphs are labeled and unweighted. To illustrate how WFSM-MaxPWS will perform in different cases, we have chosen normal distribution to reflect the general situation and negative exponential distribution for extreme cases. Dataset statistics after adding weight are given in Table 11.

[2] https://github.com/cseduashraful/graphdatasets

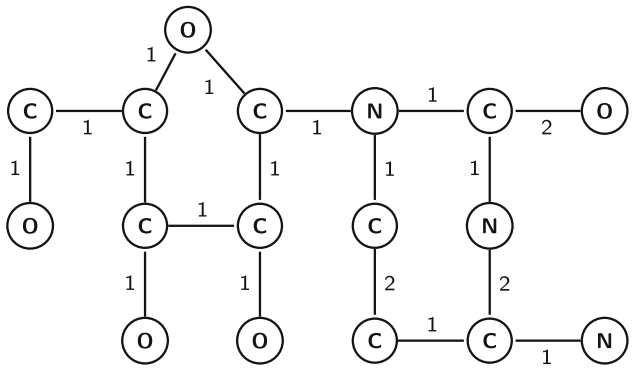


Fig. 12 A sample graph from MCF-7 dataset

To analyze how WFSM-MaxPWS performs on a different range of weights, we have given weights to the edges of MCF-7 in the range [0,1]. For P388, this range is [0,18], and for the Yeast dataset, this range is [0,3.6].

6.2.3 Performance evaluation on MCF-7

For the MCF-7 dataset with normal distribution, we have run our algorithms for threshold levels 300, 400, 500 and 600. And, for negative exponential weight distribution, used threshold levels are 250, 300, 350, and 400. Required execution time (minutes) for these threshold levels are shown in Tables 12 and 13. Threshold vs. runtime results has been plotted in Fig. 13 (a and b).

From Fig. 13(a and b), WFSM-MaxPWS is observed to take less time for both distributions than MaxPWS-gSpan and MaxW-gSpan. In the case of the negative exponential distribution, the runtime performance gain is huge compared to MaxW-gSpan. For threshold level 600, MaxW-gSpan takes 61 minutes. For the same threshold level, WFSM-MaxPWS takes 16 minutes only, which is 26.22% to that of MaxW-gSpan. The gap in runtime increases with decreasing thresholds. For threshold level 300, the runtime required by WFSM-MaxPWS decreases to 17.44% of the runtime required by MaxW-gSpan. The required runtime gap between MaxPWS-gSpan and WFSM-MaxPWS is also evident. WFSM-MaxPWS takes only 51.61% of the runtime required by MaxPWS-gSpan for threshold level 600. This behavior is obvious for negative exponential distribution because the frequency of edges having maximum weight (MaxW) is very low, and to calculate pruning measure MaxPWS, MaxPWS-gSpan uses MaxW.

This runtime improvement is basically the outcome of efficient search-space reduction of WFSM-MaxPWS. WFSM-MaxPWS generates a very small number of candidates, so the time wasted on unnecessary extension and subgraph-isomorphism checking is very low. Tables 14 and 15 show comparative lists of number of generated candidates for P388 dataset.

Table 11 Datasets for WFSM-MaxPWS

Dataset	#graphs	#distinct edge	Average Edge Count	Distribution	MinW	MaxW
MCF-7	2293	54	36	normal (μ=0.5,σ=0.07)	0.13	0.86
				negExpo (f=1)	0.07	0.98
P388	2297	64	30	normal (μ=10,σ=1.5)	3.16	16.84
				negExpo (f=18)	1.55	16.86
Yeast	9567	125	26	normal (μ=2,σ=0.3)	0.46	3.55
				negExpo (f=3.6)	0.23	3.53

We observed from Tables 14 and 15 that Extension Precision (According to (14)) is higher for WFS-MaxPWS which proves its search-space reduction efficiency. Figure 13(c and d) shows a precision comparison in the graph.

The large gap between the lines of WFSM-MaxPWS and gSpan-MaxW proves the effectiveness of our proposed canonical ordering in Fig. 13(c and d). One point to notice here is that, due to Occurrence List storage, both WFSM-MaxPWS and MaxPWS-gSpan require more memory than MaxW-gSpan. However, the required memory for the Occurrence List is very small as it is just a list of graph ids. Figure 13(e) shows a memory requirement graph for MCF-7 datasets.

6.2.4 Performance evaluation on P388

Used thresholds are [5000, 6000, 7000, 8000] and [2300, 2500, 2700, 2900] for P388 (normal) and P388 (negative exponential) respectively. For P388, WFSM-MaxPWS has also achieved significant runtime improvement. It takes only 48.07% and 19.53% of the runtimes for MaxPWS-gSpan and MaxW-gSpan, respectively, for threshold 5000. Runtime comparison graph for P388 dataset is shown in Fig. 14(a and b). We observed from Fig. 14(a and b) that, for smaller thresholds, WFSM-MaxPWS runtime improvement is huge. As the threshold increases, this gap reduces. Compared to the normal distribution, performance gain in the negative exponential distribution is enormous. For example, it takes only 8.82% and 2.81% of the runtimes for MaxPWS-gSpan and MaxW-gSpan respectively for threshold 2300 (Tables 16 and 17).

Tables 18 and 19 show the number of candidate subgraphs that were extended for each threshold level of the P388 dataset. These tables indicate the efficient search space reduction of WFSM-MaxPWS and explain the runtime improvement shown in Fig. 14(a and b). The precision graph in Fig. 14(c and d) shows it even more clearly.

Despite MCF-7 and P388 having a similar number of graphs, both MaxPWS-gSpan and MaxW-gSpan perform poorly when mining on P338 with a negative exponential distribution. The reason behind this huge performance drop is the relative individual contribution of weight in the weighted support calculation. For P388, the contribution of weight is around 16 to 18 times more when compared to that of MCF-7, as weights for MCF-7 range from 0.07 to 0.98, whereas P388's weight ranges between 1.55 and 16.86. Thus, when computing the maximum possible weight for any extension of a subgraph, poor logic affects P388 16 to 18 times more.

As the value of MaxW is affected more acutely by negative exponential weight distribution, the performance of MaxW-gSpan and MaxPWS-gSpan dropped devastatingly when negative exponential weight distribution is used. By modifying canonical ordering and using FTW in PWS calculation in (10), we have reduced the upper bound efficiently in WFSM-MaxPWS. This creates stability in performance.

P388 shows interesting behavior in memory usage. Despite storing occurrence lists, the memory requirement for WFSM-MaxPWS is slightly lower than MaxW-gSpan for the negative exponential distribution. MaxW-gSpan unnecessarily looks for larger subgraphs while performing depth-first traversal. The recursive gSpan function thus stores higher execution history in the stack. Figure 14(e) shows the

Table 12 MCF-7 runtime(normal distribution)

Threshold	WFSM-MaxPWS(m)	MaxPWS-gSpan(m)	MaxW-gSpan(m)
300	52	96	298
400	35	55	136
500	22	35	96
600	16	31	61

Table 13 MCF-7 runtime(negative exponential)

Threshold	WFSM-MaxPWS(m)	MaxPWS-gSpan(m)	MaxW-gSpan(m)
250	35	142	568
300	26	113	429
350	17	71	325
400	13	65	191

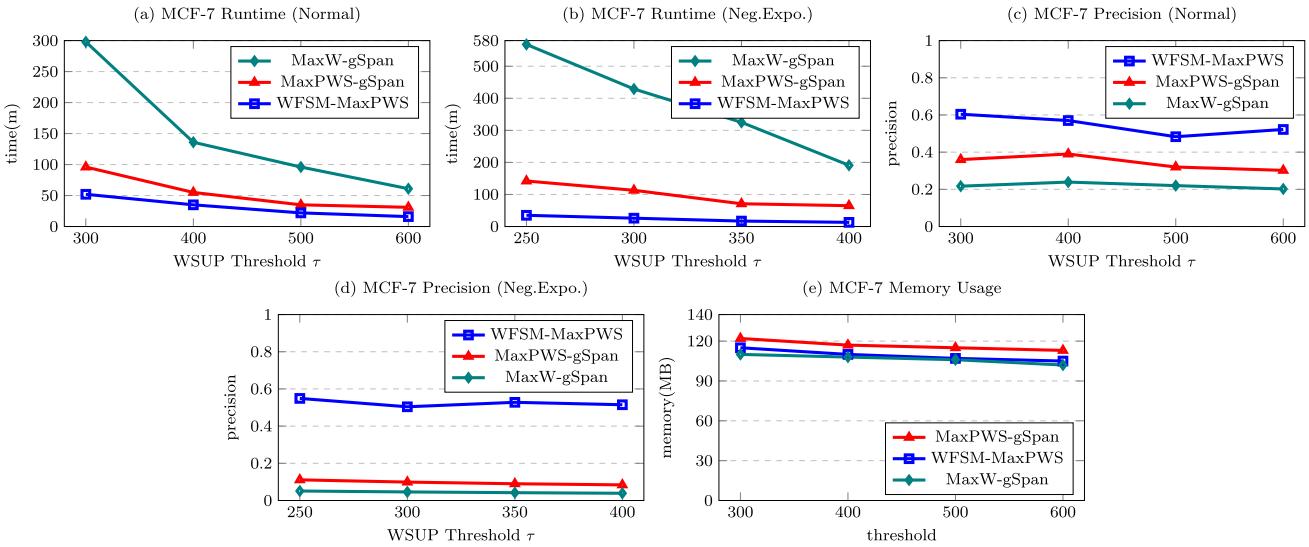


Fig. 13 Runtime, Search Space Reduction Efficiency (Precision) and Memory Usage on MCF-7 Dataset

memory requirement comparison for mining over the P388-negative exponential dataset.

6.2.5 Performance evaluation on yeast

As shown in Table 11, the Yeast dataset is comparatively larger than MCF-7 and P388. So, it is expected to take more time to mine. Graphs of the Yeast dataset are slightly smaller than the other two datasets, which somewhat reduces the runtime. Threshold levels used for Yeast datasets (normal and negative exponential) are [5500, 6000, 6500, 7000] and [750, 1000, 1250, 1500], respectively. WFSM-Max-PWS also shows superior performance for Yeast dataset in terms of runtime and search space reduction (Tables 20, 21, 22 and 23).

Figure 15(a and b) shows the required runtime for both normal and negative exponential distribution for different threshold levels. For the normal distribution, WFSM-MaxPWS takes only 77.92% and 20.27% of the runtimes for MaxPWS-gSpan and MaxW-gSpan, respectively, for threshold 5500. For negative exponential distribution, the ratios are 13.18% and 4.20% respectively for threshold 750.

The precision graph in Fig. 15(c and d) shows that normal distribution precision is more or less like MCF-7. Precision for negative exponential distribution for smaller thresholds is around 70% which resembles the search-space reduction efficiency of the WFSM-MaxPWS algorithm. For MaxW-gSpan and MaxPWS-gSpan, precision is lower than WFSM-MaxPWS significantly.

The memory requirement is slightly higher for Yeast, which is expected as Yeast is a comparatively large dataset. The change in memory requirement with the threshold level is similar to MCF-7. Figure 15(e) illustrates memory usage of yeast dataset with normal weight distribution for threshold level 5500 to 7000.

6.2.6 Scalability analysis

This section presents a scalability analysis of our proposed algorithm WFSM-MaxPWs. For scalability analysis, we have generated random weighted graph databases varying the number of graphs and average edge number for both normal and negative exponential distributions. In Fig. 16, we present the scalability analysis on the graph databases generated using the normal distribution. We have varied the number of graphs from 1,000 to 5,000 with an average number of edges of 50, and we can observe that the runtime increases

Table 14 MCF-7 Candidate Count(normal distribution)

Threshold	WFSP	WFSM-MaxPWS	MaxPWS-gSpan	MaxW-gSpan
300	426	755	1182	1963
400	220	387	564	922
500	116	240	361	529
600	71	136	235	352

Table 15 MCF-7 Candidate Count(negative exponential)

Threshold	WFSP	WFSM-MaxPWS	MaxPWS-gSpan	MaxW-gSpan
250	213	388	1916	4218
300	124	246	1241	2693
350	76	144	846	1810
400	51	99	609	1314

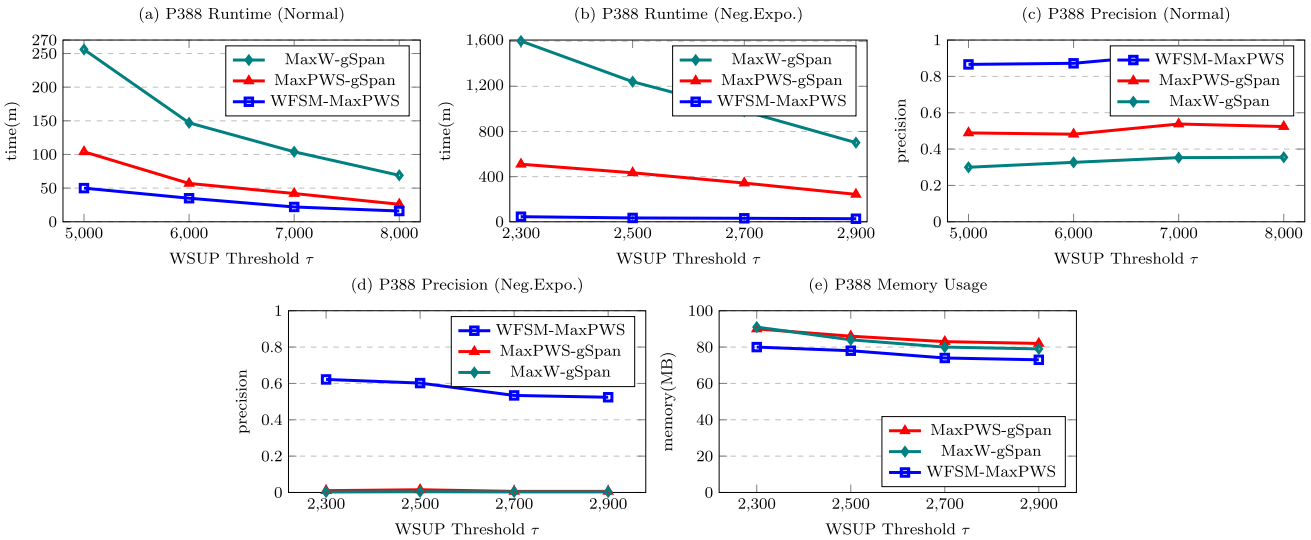


Fig. 14 Runtime, Search Space Reduction Efficiency (Precision) and Memory Usage on P388 Dataset

from 12 minutes to 247 minutes. We have also varied the average number of edges from 50 to 250 with the number of graphs of 2,000, and the runtime increases from 45 minutes to 803 minutes. In Fig. 17, we present a similar analysis on the graph databases generated using negative exponential distribution where we observe the same increasing trend in runtime with an increasing number of graphs or the average number of edges.

6.3 Performance analysis of DewgSpan

In this section, we present the experimental analysis related to DewgSpan algorithm. We have described the datasets we have utilized in Section 6.3.1, the weight inclusion technique in Section 6.3.2, and performance analysis on all the datasets in Sections 6.3.3 and 6.3.4.

6.3.1 Data description

We have used two real datasets to demonstrate the performance of the *DewgSpan* framework; they are:

– **Chemical 340**: The Chemical 340 dataset consists of 340 chemical compounds, 24 different atoms, 66 atom types, and 4 types of bonds. The average graph size in this dataset is 27 nodes and 28 edges. The largest graph in this dataset has 214 nodes and 214 edges [53].
– **NCI1**: NCI1 is a balanced dataset of chemical compounds screened for activity against non-small cell lung cancer and ovarian cancer cell lines [20]. The dataset has 4110 graphs. Graphs, on average, have 29 nodes and 33 edges, with 38 with distinct node and edge labels 37 and 3, respectively.

6.3.2 Weight inclusion

In order to reach a robust conclusion about the performance of *DewgSpan*, we varied weights on these datasets with positively skewed, normal, and negatively skewed distributions. Figure 18 shows all three weight distributions we used for performance analysis with the Chemical 340 dataset.

6.3.3 Performance evaluation on chemical 340

Figure 19a shows how efficiently *DewgSpan* has reduced search space during weighted frequent subgraph discovery in the Chemical 340 dataset with normal distribution.

Table 16 P388 runtime(normal distribution)

Threshold	WFSM-MaxPWS(m)	MaxPWS-gSpan(m)	MaxW-gSpan(m)
5000	50	104	256
6000	35	57	147
7000	22	42	104
8000	16	26	69

Table 17 P388 runtime(negative exponential)

Threshold	WFSM-MaxPWS(m)	MaxPWS-gSpan(m)	MaxW-gSpan(m)
2300	45	510	1601
2500	34	434	1241
2700	31	343	980
2900	27	243	701

Table 18 P388 Candidate Count(normal distribution)

Threshold	WFSP	WFSM-MaxPWS	MaxPWS-gSpan	MaxW-gSpan
5000	616	711	1259	2053
6000	362	415	750	1104
7000	257	278	478	739
8000	173	199	330	487

Table 20 Yeast runtime(normal distribution)

Threshold	WFSM-MaxPWS(m)	MaxPWS-gSpan(m)	MaxW-gSpan
5500	60	77	296
6000	54	68	223
6500	51	62	175
7000	47	59	158

The numbers of generated candidate patterns are plotted on the Y-axis to discover a certain number of weighted frequent subgraphs(X-axis). Figure 19a shows that generated candidate patterns are significantly lower for $DewgSpan$ compared to $MaxW-gSpan$. Reducing search space leads to lower execution time as subgraph extension and isomorphism checking will be performed on fewer graphs. In Fig. 19b, we present a runtime comparison among $MaxW-gSpan$ and $DewgSpan$ with two different bin_count values (6 and 11) which fall in line with search space comparison. We have also executed $UGMINE$ [37], an algorithm proposed for utility-based subgraph mining, on the datasets. In the comparison presented in Fig. 19b, we observed that the weaker pruning technique utilized in UGMINE resulted in a much higher runtime than WFSM-MaxPWS. Although the change in bin_count is obscure in Fig. 19b, we can clearly see the effect of the number of bins used during WDT construction in Fig. 19c where we kept weighted support threshold(τ) fixed (150) and varied bin_count from 1 to 11. The result suggests that $bin_count = 6$ to 8 executes faster than very high or very low bin_count. The runtime decreases from 125 seconds to 109 seconds for bin_count value from 1 to 7. However, the runtime rises to 113 seconds for $bin_count = 11$.

Figures 20 and 21 show experimental results gathered from the Chemical 340 dataset with positively skewed and negatively skewed weight distribution, respectively. From the figures, it is easily perceivable that search space reduction for positively skewed distribution (Fig. 20a) is comparatively better than we gained in the case of normal distribution. On the other hand, candidate count in $MaxW-gSpan$ and $DewgSpan$ is very close for negatively skewed weight distribution Fig. 20a. The factor that is responsible for it is the shape of weight distribution; for negatively skewed weight distribution (Fig. 18c), most of the edge-weight lies very close to $MaxW$ which results in a lower margin between $MaxW \times frequency$ and $MaxPWS$ and the situation is quite the opposite for positively skewed weight distribution. The same logic works behind runtime performance for negatively and positively skewed weight distribution shown in Figs. 20b and 21b. Performance regarding the number of bins used in WDT construction suggests that comparatively higher bin_count gives better performance if weights are close to $MaxW$.

6.3.4 Performance evaluation on NCI1

Figures 22, 23 and 24 show performance analysis on NCI1 dataset with normal, positively skewed and negatively skewed weight distribution, respectively. Search space reduction is significantly larger in all three cases compared to the chemical 340 dataset. The main factor that is working behind it is the size of the dataset. NCI1 is a comparatively large dataset, and experiments are conducted at a pretty low threshold level. As a result, $MaxW$ pruning measure kept extending the candidate pattern having the actual weighted support is much lower than $MaxW \times support$. On the other, efficient calculation of the $MaxPWS$ pruning measure has made it possible to keep search space in control. Runtime analysis graph (Figs. 22b, 23b and 24b) also complies with the analysis for search space comparison. And, in search of a suitable number of bins to be used with the NCI1 dataset for WDT construction, our conducted experiments varying bin_Count from 1 to 11 show that for normal and negatively skewed dis-

Table 19 P388 Candidate Count(negative exponential)

Threshold	WFSP	WFSM-MaxPWS	MaxPWS-gSpan	MaxW-gSpan
2300	248	399	30869	91053
2500	192	319	12148	38312
2700	142	266	7145	27944
2900	111	212	4953	22455

Table 21 Yeast runtime(negative exponential)

Threshold	WFSM-MaxPWS(m)	MaxPWS-gSpan(m)	MaxW-gSpan
750	103	781	2449
1000	61	487	1533
1250	49	354	1265
1500	38	275	1054

Table 22 Yeast Candidate Count(normal distribution)

Threshold	WFSP	WFSM-MaxPWS	MaxPWS-gSpan	MaxW-gSpan
5500	122	193	298	506
6000	101	156	255	414
6500	81	129	222	342
7000	74	105	183	290

tribution $bin_Count = 7$ gives a better result (Figs. 22c, 24c) and for the positively skewed dataset, as weights are shifted towards the lower range, $bin_Count = 5$ gives a better result (Fig. 23c). The fact that the margin between $DewgSpan$ and $MaxW-gSpan$ gets larger in terms of search space and runtime performance when the dataset gets larger (chemical 340 to NCI1) proves the scalability of our proposed approach.

6.4 Comparison between WFSM-MaxPWS and DewgSpan on static weighted graphs

As we have stated earlier, WFSM-MaxPWS only applies to graphs with static weighted edges, whereas DewgSpan can be used for both dynamic and static weighted edges. In this section, we have shown experimental results gathered after applying WFSM-MaxPWS and $DewgSpan$ to demonstrate how they perform compared to each other. Here, we have used two real datasets, MCF7 and P388, with normal and negative exponential weight distribution. We have used static weights only for edges. MCF7 dataset has 2293 graphs with an average of 36 edges in each graph. The entire dataset has a total of 54 distinct edges. Edge weights follow normal distribution ($\mu = 0.47, \sigma = 0.1, MaxW = 0.865, MinW = 0.134$). On the other hand, the P388 dataset has 2297 graphs with an average of 30 edges in each graph and 64 distinct edges. Edge weights of P388 dataset follow negative exponential distribution ($\lambda = 1.8, scale = 5, MaxW = 16.86, MinW = 1.55$).

Figure 25(a, b, c and d) shows experimental results comparing $WFSM-MaxPWS$ and $DewgSpan$ on MCF7 and P388 dataset respectively. If we look at search space comparison graphs (Fig. 25(a and c)) of these two datasets, it is evident that $WFSM-MaxPWS$ performs better. $WFSM-MaxPWS$ proposed modified canonical ordering of subgraph DFScode which allows using FTW in $MaxPWS$ calculation instead of $MaxW$. This modification is not applicable for graphs with dynamic edge weights, as edge weights vary in different occurrences and as $DewgSpan$ needs to be compatible with a dynamic edge-weighted graph, it can not prune as many patterns as $WFSM-MaxPWS$. We can see the reflection of the search space reduction performance of WFSM-MaxPWS in Fig. 25(b and d) generated using runtime time of MCF7 and P388, respectively, for different threshold levels.

Table 23 Yeast Candidate Count(negative exponential)

Threshold	WFSP	WFSM-MaxPWS	MaxPWS-gSpan	MaxW-gSpan
750	342	506	7382	27382
1000	177	289	4335	17724
1250	115	172	2932	6052
1500	75	116	2115	4410

From the above discussion and findings of performance evaluation, we can say that using $MaxPWS$ anti-monotonic pruning measure, $WFSM-MaxPWS$ and $DewgSpan$ both are capable of discovering all the weighted frequent substructures in a graph database significantly faster than existing $MaxW$ anti-monotonic pruning measure. And, even though $DewgSpan$ is compatible with static weighted graph database mining, minor but effective redefinition of canonical ordering coupled with FTW based formula for $MaxPWS$ calculation makes $WFSM-MaxPWS$ way faster in this regard.

7 Conclusions

In this article, we have proposed two frameworks for weighted graph mining called $WFSM-MaxPWS$ and $DewgSpan$. $WFSM-MaxPWS$ is designed to focus on performance improvement for $WFSM$ on graphs with static edge weight, whereas $DewgSpan$ applies to graphs with dynamic edge weight. A novel pruning measure named MaxPWS contributes to this performance improvement by reducing the search space. We have provided theoretical proofs and empirical analysis that validate the superiority of the MaxPWS pruning measure from the perspective of search-space reduction while ensuring completeness. Experimental results on multiple datasets with different weight distributions indicate significant improvement in the mining time compared to existing weighted graph mining frameworks. As future work, $WFSM-MaxPWS$ and $DewgSpan$ can be extended for closed-weighted and uncertain substructure mining with minor modifications. In real-life applications, the edge weights or even edges vary over time. Mining weighted patterns from such temporal weighted databases efficiently can be explored. Mining weighted patterns from spatial graphs can also be considered. Devising efficient pruning techniques or approximate methods can be developed for faster mining to facilitate real-time applications. Using weighted frequent patterns as features can be considered for improving the outcomes of learning tasks such as embedding, classification, and clustering.

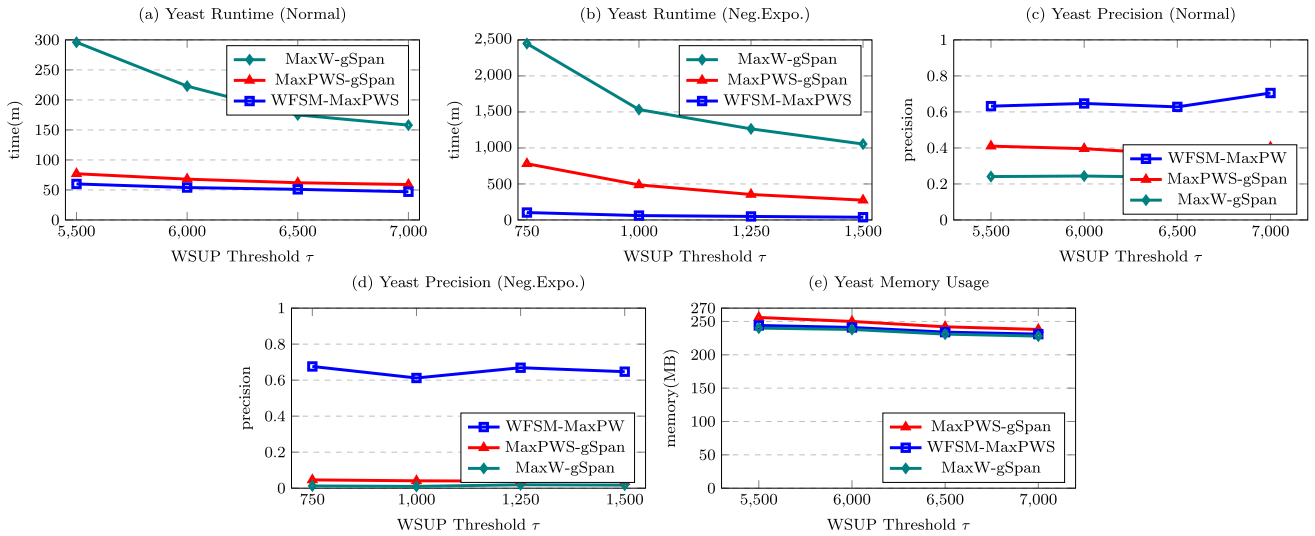


Fig. 15 Runtime, Search Space Reduction Efficiency (Precision) and Memory Usage on Yeast Dataset

Fig. 16 Scalability analysis of WFSM-MaxPWs(Normal Distribution, $\mu = 0.5, \sigma = 0.1$)

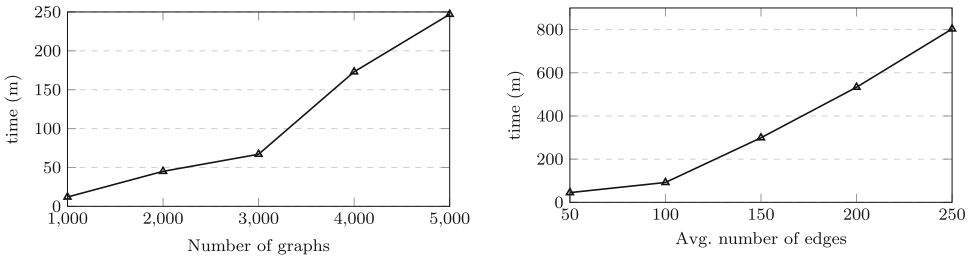
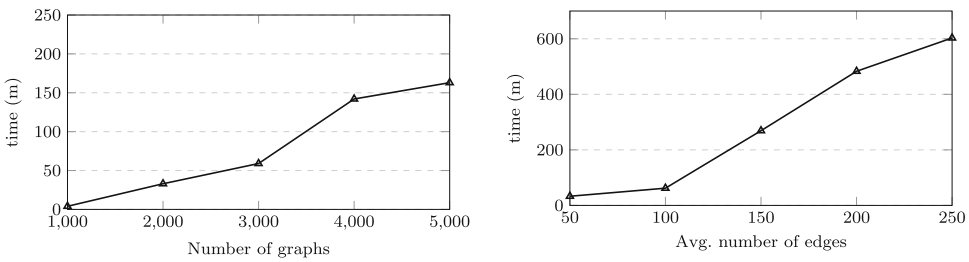
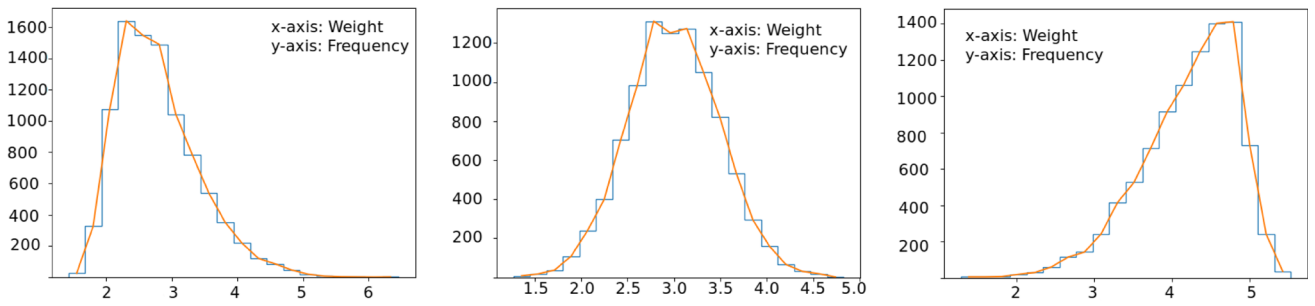


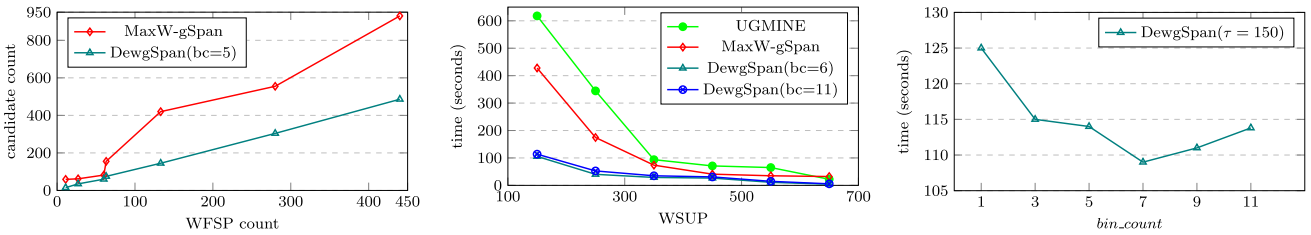
Fig. 17 Scalability analysis of WFSM-MaxPWs(Negative Exponential Distribution, $f = 1$)





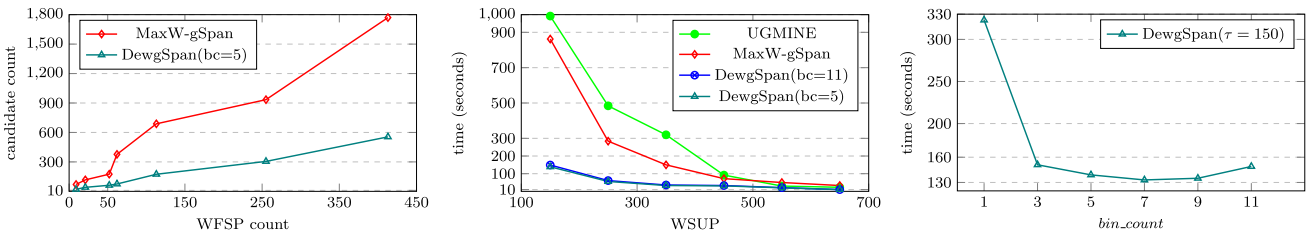
(a) Positively Skewed (b) Normal (c) Negatively Skewed

Fig. 18 Chemical 340 Weight Distribution



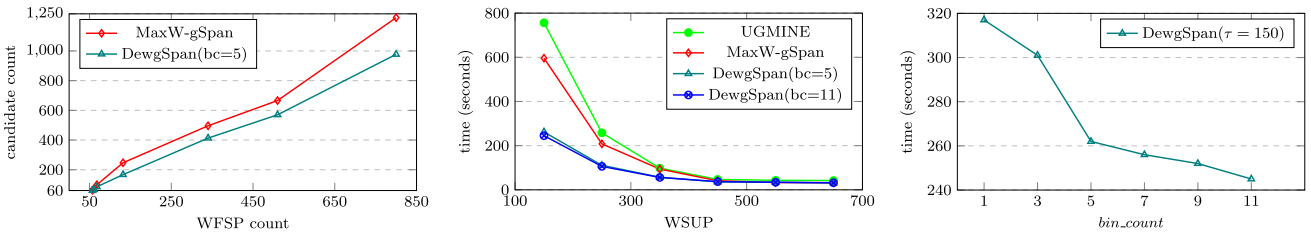
(a) Search Space (b) Time vs Threshold (c) Time vs *bin_Count*

Fig. 19 Experimental Results on Chemical 340 (Normal Distribution, $\mu = 3, \sigma = 0.5$)



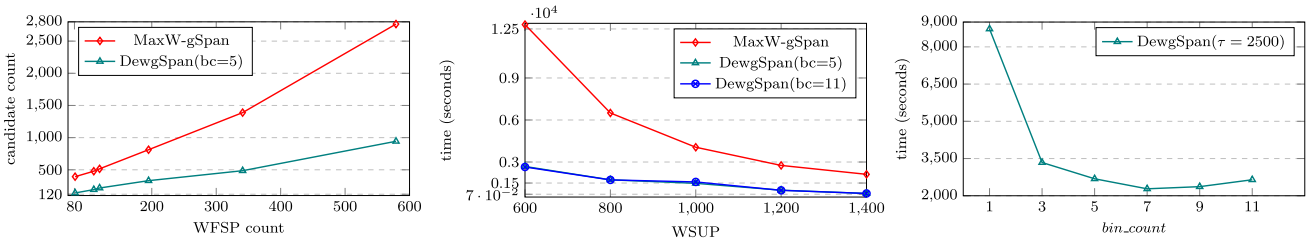
(a) Search Space (b) Time vs Threshold (c) Time vs *bin_Count*

Fig. 20 Experimental Results on Chemical 340 (Positively Skewed Distribution, $skewness = 5, loc = 2, scale = 1$)



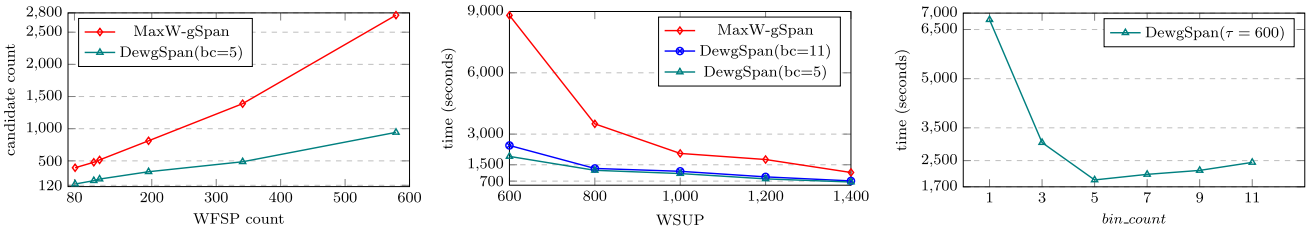
(a) Search Space (b) Time vs Threshold (c) Time vs *bin_Count*

Fig. 21 Experimental Results on Chemical 340 (Negatively Skewed Distribution, $skewness = -5, loc = 5, scale = 1$)



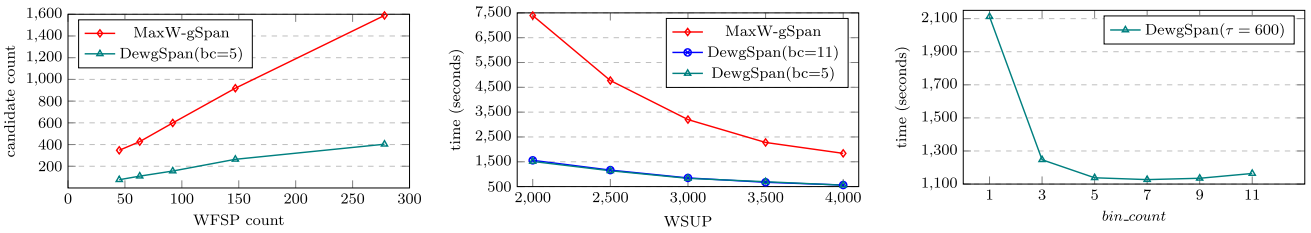
(a) Search Space (b) Time vs Threshold (c) Time vs *bin_Count*

Fig. 22 Experimental Results on NCI1 (Normal Distribution, $\mu = 0.7, \sigma = 0.15$)



(a) Search Space (b) Time vs Threshold (c) Time vs *bin_Count*

Fig. 23 Experimental Results on NCI1 (Positively skewed Distribution, $skewness = 5$, $loc = 0.5$, $scale = 0.3$)



(a) Search Space (b) Time vs Threshold (c) Time vs *bin_Count*

Fig. 24 Experimental Results on NCI1 (Negatively Skewed Distribution, $skewness = -5$, $loc = 1.2$, $scale = 0.5$)

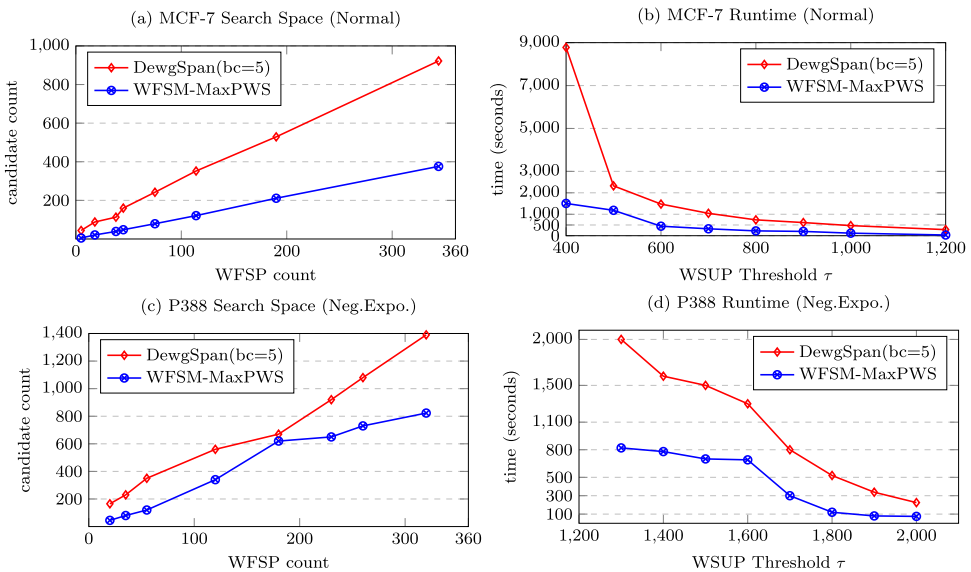


Fig. 25 WFSM-MaxPWS vs DewgSpan Performance Evaluation

Acknowledgements Out of the two frameworks discussed in this article, a preliminary version of the framework for static edge-weighted substructure mining has been previously published in PAKDD 2018 [50].

Author Contributions Conceptualization: Md. Ashraful Islam & Chowdhury Farhan Ahmed **Methodology:** Md. Ashraful Islam **Formal analysis and investigation:** Md. Ashraful Islam, Chowdhury Farhan Ahmed & Md. Tanvir Alam **Writing - original draft preparation:** Md. Ashraful Islam & Md. Tanvir Alam **Writing - review and editing:** Chowdhury Farhan Ahmed & Carson Kai-Sang Leung **Supervision:** Chowdhury Farhan Ahmed & Carson Kai-Sang Leung

Funding This work is partially supported by (a) University of Dhaka, (b) Natural Sciences and Engineering Research Council of Canada (NSERC), and (c) University of Manitoba.

Data Availability The data that support the finding of this study is available at https://github.com/cseduashraful/graphdatasets

Declarations

Competing Interests The authors have no competing interests to declare that are relevant to the content of this article.

Ethical and informed consent for data used: Used data are open-source and have no associated privacy and copyright issues.

References

1. Agrawal R, Srikant R (1994) Fast algorithms for mining association rules in large databases. In: VLDB'94, Proceedings of 20th international conference on very large data bases, September 12-15, 1994, Santiago de Chile, Chile, pp 487–499 (1994)
2. Han J, Pei J, Yin Y (2000) Mining frequent patterns without candidate generation. In: SIGMOD '00: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, pp 1–12. ACM
3. Han J, Pei J, Mortazavi-Asl B, Pinto H, Chen Q, Dayal U, Hsu M (2001) Prefixspan: Mining sequential patterns efficiently by prefix-projected pattern growth. In: Proceedings of the 17th international conference on data engineering, pp 215–224
4. Islam MA, Rafi MR, Azad Aa, Ovi JA (2022) Weighted frequent sequential pattern mining. Appl Intell 52(1):254–281
5. Nguyen H, Le T, Nguyen M, Fournier-Viger P, Tseng VS, Vo B (2022) Mining frequent weighted utility itemsets in hierarchical quantitative databases. Knowledge-Based Systems 237:107709
6. Roy KK, Moon MHH, Rahman MM, Ahmed CF, Leung CK (2021) Mining sequential patterns in uncertain databases using hierarchical index structure. In: Advances in knowledge discovery and data mining: 25th Pacific-Asia Conference, PAKDD 2021, Virtual Event, May 11–14, 2021, Proceedings, Part II, Springer, pp 29–41
7. Leung CKS, Tanbeer SK (2013) PUF-tree: a compact tree structure for frequent pattern mining of uncertain data. In: Pacific-Asia Conference on Knowledge Discovery and Data Mining, Springer, pp 13–25
8. Wang J, Liu C, Fu X, Luo X, Li X (2019) A three-phase approach to differentially private crucial patterns mining over data streams. Computers & Security 82:30–48
9. Tsuda K, Kudo T (2006) Clustering graphs by weighted substructure mining. In: Proceedings of the 23rd international conference on Machine learning, ACM, pp 953–960
10. Cheng Z, Flouvat F, Selmaoui-Folcher N (2017) Mining recurrent patterns in a dynamic attributed graph. In: Proceedings of 21st Pacific-Asia conference on knowledge discovery and data mining (PAKDD 2017), Part II, pp 631–643
11. Huang Z, Ye Y, Li X, Liu F, Chen H (2017) Joint weighted nonnegative matrix factorization for mining attributed graphs. In: Proceedings of 21st Pacific-asia conference on knowledge discovery and data mining (PAKDD 2017), Part I, pp 368–380
12. Khan A, Akcora CG (2022) Graph-based management and mining of blockchain data. In: Proceedings of the 31st ACM international conference on information & knowledge management, pp 5140–5143
13. Ning B, Sun Y, Tao X, Li G (2021) Differential privacy protection on weighted graph in wireless networks. Ad hoc networks 110:102303
14. Gu Z, Liu H, Feng S (2022) Diversity-induced consensus and structured graph learning for multi-view clustering. Appl Intell pp 1–15
15. Li K, Ye W (202) Semi-supervised node classification via graph learning convolutional neural network. Appl Intell pp 1–13
16. Ju W, Qin Y, Qiao Z, Luo X, Wang Y, Fu Y, Zhang M (2022) Kernel-based substructure exploration for next poi recommendation. In: 2022 IEEE International conference on data mining (ICDM), IEEE, pp 221–230
17. Zhang Z, Bu J, Ester M, Li Z, Yao C, Yu Z, Wang C (2021) H2MN: Graph similarity learning with hierarchical hypergraph matching networks. In: Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining, pp 2274–2284
18. Yan X, Han J (2002) gspan: Graph-based substructure pattern mining. In: 2002 IEEE International conference on data mining, 2002. Proceedings., IEEE, pp 721–724
19. Nijssen S, Kok JN (2005) The gaston tool for frequent subgraph mining. Electronic Notes in Theoretical Computer Science 127(1):77–87
20. Nguyen D, Luo W, Nguyen TD, Venkatesh S, Phung D (2018) Learning graph representation via frequent subgraphs. In: Proceedings of the 2018 SIAM International Conference on Data Mining, SIAM, pp 306–314
21. Alam MT, Ahmed CF, Samiullah M, Leung CK (2021) Discriminating frequent pattern based supervised graph embedding for classification. In: Advances in knowledge discovery and data mining, pp 16–28
22. Nowozin S, Tsuda K, Uno T, Kudo T, BakIr G (2007) Weighted substructure mining for image analysis. In: 2007 IEEE Conference on computer vision and pattern recognition, IEEE, pp 1–8
23. Henderson TA, Podgurski A (2018) Behavioral fault localization by sampling suspicious dynamic control flow subgraphs. In: 2018 IEEE 11th International conference on software testing, verification and validation (ICST), IEEE, pp 93–104
24. Salehi Z, Ghiasi M, Sami A (2012) A miner for malware detection based on API function calls and their arguments. In: Artificial intelligence and signal processing (AISP), 2012 16th CSI International Symposium on, IEEE, pp 563–568
25. Du Y, Wang J, Li Q (2017) An android malware detection approach using community structures of weighted function call graphs. IEEE Access 5:17478–17486
26. Lakhotia A, Preda MD, Giacobazzi R (2013) Fast location of similar code fragments using semantic'juice'. In: Proceedings of the 2nd ACM SIGPLAN program protection and reverse engineering workshop, ACM, pp 5
27. Ahmed CF, Tanbeer SK, Jeong BS, Lee YK, Choi HJ (2012) Single-pass incremental and interactive mining for weighted frequent patterns. Expert Systems with Applications 39(9):7976–7994
28. Zou Z, Li J, Gao H, Zhang S (2010) Mining frequent subgraph patterns from uncertain graph data. IEEE Transactions on Knowledge and Data Engineering 22(9):1203–1218

29. Bogdanov P, Mongiovì M, Singh AK (2011) Mining heavy subgraphs in time-evolving networks. In: 2011 IEEE 11th International conference on data mining, IEEE, pp 81–90
30. Rozenshtein P, Gionis A (2019) Mining temporal networks. In: Proceedings of the 25th ACM SIGKDD International conference on knowledge discovery & data mining, ACM, pp 3225–3226
31. Petelin B, Kononenko I, Malačič V, Kukar M (2019) Frequent subgraph mining in oceanographic multi-level directed graphs. Int J Geographical Inf Sci 1–24
32. Gong Y, Jia L (2019) Research on SVM environment performance of parallel computing based on large data set of machine learning. J Supercomput 1–18
33. Eichinger F, Böhm K, Huber M (2008) Mining edge-weighted call graphs to localise software bugs. In: Joint european conference on machine learning and knowledge discovery in databases, Springer, pp 333–348
34. Jiang C, Coenen F (2008) Graph-based image classification by weighting scheme. In: International conference on innovative techniques and applications of artificial intelligence, Springer, pp 63–76
35. Shinoda M, Ozaki T, Ohkawa T (2009) Weighted frequent subgraph mining in weighted graph databases. In: 2009 IEEE International conference on data mining workshops, IEEE, pp 58–63
36. Ozaki T, Etoh M (2011) Closed and maximal subgraph mining in internally and externally weighted graph databases. In: Proceedings of the 2011 IEEE International conference on advanced information networking and applications (AINA 2011) Workshops, IEEE, pp 626–631
37. Alam MT, Roy A, Ahmed CF, Islam MA, Leung CK (2023) UGMINE: utility-based graph mining. Applied Intelligence 53(1):49–68
38. Eichinger F, Huber M, Böhm K (2010) On the usefulness of weight-based constraints in frequent subgraph mining. In: SGAI Conf., Springer, pp 65–78
39. Jiang C, Coenen F, Zito M (2010) Frequent sub-graph mining on edge weighted graphs. In: International conference on data warehousing and knowledge discovery, Springer, pp 77–88
40. Jiang C, Coenen F, Zito M (2010) Finding frequent subgraphs in longitudinal social network data using a weighted graph mining approach. Adv Data Mining Appl 405–416
41. Elsayed A, Coenen F, Jiang C, Garcia-Finana M, Sluming V (2010) Corpus callosum mr image classification. Knowledge-Based Systems 23(4):330–336
42. Jiang C, Coenen F, Sanderson R, Zito M (2010) Text classification using graph mining-based feature extraction. In: Research and Development in Intelligent Systems XXVI, Springer, pp 21–34
43. Lee G, Yun U (2012) Mining weighted frequent sub-graphs with weight and support affinities. In: International workshop on multi-disciplinary trends in artificial intelligence, Springer, pp 224–235
44. Lee G, Yun U, Kim D (2016) A weight-based approach: frequent graph pattern mining with length-decreasing support constraints using weighted smallest valid extension. Advanced Science Letters 22(9):2480–2484
45. Babu N, John A (2016) A distributed approach to weighted frequent subgraph mining. In: International conference on on emerging technological trends [ICETT], IEEE, pp 1–7
46. Gupta A, Thakur H, Gupta T, Yadav S (2017) Regular pattern mining (with jitter) on weighted-directed dynamic graphs. Journal of Engineering Science and Technology 12(2):349–364
47. Le NT, Vo B, Nguyen LB, Fujita H, Le B (2020) Mining weighted subgraphs in a single large graph. Information Sciences 514:149–165
48. Le NT, Vo B, Nguyen LB, Le B (2022) OWGraMi: Efficient method for mining weighted subgraphs in a single graph. Expert Syst Appl 117625
49. Ashraf N, Haque RR, Islam M, Ahmed CF, Leung CK, Mai JJ, Wodi BH et al (2019) WeFreS: weighted frequent subgraph mining in a single large graph. In: Industrial conference on data mining. ibai publishing
50. Islam MA, Ahmed CF, Leung CK, Hoi CS (2018) WFSM-MaxPWS: an efficient approach for mining weighted frequent subgraphs from edge-weighted graph databases. In: Pacific-Asia conference on knowledge discovery and data mining, Springer, pp 664–676
51. Zaki MJ, Meira W (2014) Data Mining and Analysis: Fundamental Concepts and Algorithms. Cambridge University Press, New York, NY, USA
52. Yan X Graph datasets. http://www.cs.ucsb.edu/~xyan/dataset.htm
53. Mehmood D, Shafiq B, Vaidya J, Hong Y, Adam N, Atluri V (2012) Privacy-preserving subgraph discovery. In: IFIP Annual conference on data and applications security and privacy, Springer, pp 161–176

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Md. Ashraful Islam received his B.Sc.(Hons.) and M.S. degrees in Computer Science and Engineering from University of Dhaka, Bangladesh in 2015 and 2017 respectively. He is currently working as an Assistant Professor in the Department of CSE, University of Dhaka, Bangladesh. His current research interests include Data Mining, Temporal Structured Data Learning and ML system



Chowdhury Farhan Ahmed received the BS and MS degrees in Computer Science from the University of Dhaka, Bangladesh in 2000 and 2002 respectively, and the PhD degree in Computer Engineering from the Kyung Hee University, South Korea in 2010. He was a postdoctoral research fellow in the ICube Laboratory, University of Strasbourg, France from 2013 to 2015. He worked as a visiting scholar in the Faculty of Engineering and Information Technology, University of Technology Sydney, Australia in March, 2019. Since 2004, he has been working as a faculty member (and as a Professor since 2016) in the Department of Computer Science and Engineering, University of Dhaka, Bangladesh. His research interests are in the areas of data mining, knowledge discovery, data science and machine learning.



Md. Tanvir Alam received his B.Sc.(Hons.) and M.S. degree in Computer Science and Engineering from the Department of Computer Science and Engineering, University of Dhaka. He is currently working as a lecturer in the Department of Computer Science and Engineering, University of Dhaka. His research interests are graph neural networks, graph mining.



Carson Kai-Sang Leung received his B.Sc.(Hons.), M.Sc. and Ph.D. degrees, all in computer science, from the University of British Columbia, Vancouver, Canada. He is currently a Professor at the University of Manitoba, Canada. He has contributed more than 400 refereed publications on the topics of applied intelligence, artificial intelligence, big data, bioinformatics, data analytics, data mining, data science, health informatics, machine learning, social network analysis, and visual analytics. These include publications in international journals and conferences such as *ACM Transactions on Database Systems (TODS)*, *ACM Transactions on Knowledge Discovery from Data (TKDD)*, IEEE ICDE, IEEE ICDM, and PAKDD. Moreover, he has served as the Editor-in-Chief for *Advances in Data Science and Adaptive Analysis* and an Associate Editor for journals like Springer's *Social Network Analysis and Mining (SNAM)*. He has served on the Organizing Committee of the ACM CIKM, ACM SIGMOD, IEEE DSAA, IEEE ICDM, and other conferences; he has also served as a PC member of numerous conferences including ACM KDD, ECML/PKDD and PAKDD. He is a Senior Member of both the ACM and the IEEE; he is also an IEEE Computer Society Distinguished Contributor.